
Sr. Agim

Agente SDR Imobiliário com Inteligência Artificial Generativa

Atendimento, qualificação, agendamento e follow-up de leads com agentes especializados

ALUNO	Fernando Monin
CURSO	Pós-Tech – Inteligência Artificial para Developers
INSTITUIÇÃO	FIAP
REPOSITÓRIO	https://github.com/fmonin/tech-fase5-agente-sdr-imobiliario-ia.git
VÍDEO	https://youtu.be/J-EbXafZE1Q

Fernando Monin – rm369303

Pós-graduação em Inteligência Artificial para Desenvolvedores

São Paulo – 2026

Sumário

1. Resumo executivo

2. Contexto e problema

- 2.1 O papel do SDR na imobiliária
- 2.2 Por que a velocidade de resposta importa
- 2.3 A jornada do cliente: antes e depois

3. Objetivos

- 3.1 Objetivo geral
- 3.2 Objetivos específicos

4. Visão geral da solução

- 4.1 Proposta de valor e indicadores de negócio
- 4.2 Regras de negócio implementadas
- 4.3 Custo estimado de operação

5. Arquitetura e fluxo de execução

- 5.1 Passo a passo do processamento
- 5.2 Estado da conversa
- 5.3 Regras de roteamento do grafo
- 5.4 Modelo de dados
- 5.5 Algoritmos principais
- 5.6 Clean Architecture e princípios SOLID
- 5.7 Tecnologias
- 5.8 Configuração e segurança
- 5.9 Observabilidade

6. Soluções implementadas por módulo

- 6.1 Atendimento ao cliente
- 6.2 Investimento
- 6.3 Captação de imóveis
- 6.4 Follow-up, pós-visita e encerramento
- 6.5 Área do Corretor
- 6.6 Painel da Imobiliária
- 6.7 Canal Telegram
- 6.8 Voz

7. Modelos utilizados

8. Preparação dos dados e engenharia de prompt

- 8.1 Base de imóveis e dados de mercado
- 8.2 Embeddings e RAG
- 8.3 Engenharia de prompt
- 8.4 Proteção de dados no contexto do modelo

9. Resultados e avaliação

- 9.1 Critérios de avaliação do Tech Challenge

10. Pontos importantes da solução

11. Pontos críticos, limitações e riscos

- 11.1 Plano de implantação em fases
- 11.2 LGPD e responsabilidade

12. Testes e qualidade

- 12.1 Problemas encontrados e resolvidos

13. Como executar o projeto

- 13.1 Pré-requisitos

- 13.2 Instalação local
- 13.3 Interface Streamlit
- 13.4 Bot do Telegram
- 13.5 Docker
- 13.6 Testes e diagnóstico

14. Conclusão

15. Referências

- 15.1 Documentação do projeto
- 15.2 Referências técnicas

Anexo A — Decisões de projeto

Anexo B — Estrutura de pastas

Anexo C — Mapa de componentes

1. Resumo executivo

Este trabalho apresenta o **Sr. Agim**, uma Prova de Conceito de um **Agente SDR (Sales Development Representative) para imobiliárias** construído com Inteligência Artificial Generativa. O agente faz o primeiro atendimento do cliente pelo navegador ou pelo Telegram, em texto ou por voz: identifica quem está falando, entende o que a pessoa procura, sugere imóveis reais da base, agenda a visita com o corretor certo e entrega ao corretor um resumo pronto.

A ideia principal foi construir um agente que soasse humano **sem inventar informação**. Para isso, o atendimento foi dividido entre vários agentes especializados, coordenados por um grafo LangGraph, o código foi organizado em **Clean Architecture seguindo os cinco princípios SOLID**, e foi adotada uma regra simples: **o Python decide, o LLM conversa**. Preços, horários, imóveis, cálculos e a escolha do corretor saem do código; o modelo de linguagem (Azure OpenAI **gpt-4.1-mini**) só interpreta a mensagem e redige a resposta.

Além do que o desafio pede, o projeto cobre etapas que costumam ficar para trás no dia a dia de uma imobiliária: follow-up de quem parou de responder, pós-visita que realimenta as sugestões, captação de imóveis de proprietários com estimativa de valor, uma Área do Corretor com carteira e funil, e um Painel da Imobiliária para o gestor.

Resultado da entrega

Foi criado um agente funcional de ponta a ponta, com dois canais (web e Telegram), voz, 11 agentes no grafo, memória entre sessões e canais, três telas para três públicos, **240 testes automatizados**, execução local sem nenhuma chave (modo simulado), suporte a Docker e integração com serviços Azure.

9 / 9 objetivos do desafio atendidos	3 / 3 cenários de referência cobertos	11 agentes no grafo LangGraph	240 testes automatizados passando
2 + voz canais: web e Telegram, com áudio	41 imóveis na base simulada	9 corretores com zonas de atuação	21 decisões técnicas documentadas

2. Contexto e problema

O mercado imobiliário vem passando por uma transformação puxada por inteligência artificial, automação e atendimento conversacional. Ainda assim, toda imobiliária conhece a mesma cena: o anúncio gera contatos, mas boa parte deles esfria antes de alguém responder. Os motivos se repetem:

- **Demora na resposta.** O cliente escreve à noite ou no fim de semana e só é atendido no dia seguinte, quando já falou com outras imobiliárias.
- **Falta de acompanhamento.** Quem para de responder raramente recebe um segundo contato.
- **Corretor ocupado com triagem.** Boa parte do dia vai para perguntas repetidas ("qual bairro?", "quantos quartos?", "até quanto?").
- **Informação que se perde.** O que o cliente contou não chega organizado a quem vai atendê-lo na visita.
- **Gestão sem visão de funil.** Ninguém sabe ao certo quantos contatos viram visita, quantas visitas viram proposta e onde falta imóvel para a demanda que chega.

O Hackathon da Fase 5 propõe atacar esse problema com uma POC de Agente SDR Imobiliário, trazendo três cenários de referência: **compra** ("estou procurando apartamento na zona sul"), **investimento** ("quero investir em imóveis para renda") e **follow-up** (o cliente para de responder e o agente retoma de onde parou).

A solução não pretende substituir o corretor. Ela cuida da parte repetitiva do atendimento, 24 horas por dia, e entrega ao corretor humano o cliente já qualificado, com visita marcada e um resumo da conversa. A negociação e o fechamento continuam com quem conhece o imóvel e o cliente. Esse limite foi considerado desde o início, porque um agente automático não deve fechar negócio nem dar garantias em nome da imobiliária.

2.1 O papel do SDR na imobiliária

Em vendas, o **SDR (Sales Development Representative)** é quem recebe o contato novo, faz as primeiras perguntas, separa quem tem interesse real de quem está só pesquisando e passa adiante os clientes prontos para conversar com quem fecha o negócio. Numa imobiliária, essa função costuma ficar com o próprio corretor ou com uma recepção sobrecarregada, o que gera dois problemas: o corretor gasta horas com triagem e os contatos que chegam fora do horário comercial esperam.

Etapas do funil	Quem faz hoje	Com o Sr. Agim
Primeiro contato	Corretor ou recepção, no horário comercial	Agente, em segundos, 24 horas por dia, na web ou no Telegram
Qualificação	Perguntas repetidas a cada cliente	Agente pergunta só o que falta e classifica o lead (frio, morno, quente)
Sugestão de imóveis	Corretor procura na base ou no portal	Agente busca na base e mostra até três opções que cabem no perfil
Agendamento	Troca de mensagens até achar um horário	Agente marca com o corretor da região e evita conflito e duplicidade
Repassa ao corretor	Anotações soltas ou nada	Resumo pronto com perfil, imóveis vistos e horário combinado
Acompanhamento	Depende da memória de cada um	Follow-up e pós-visita automáticos, com limite e opt-out
Negociação e fechamento	Corretor	Corretor (o agente não negocia)

2.2 Por que a velocidade de resposta importa

O valor de um lead cai rapidamente com o tempo. Um estudo publicado na *Harvard Business Review* ("The Short Life of Online Sales Leads", 2011), que analisou cerca de 1,25 milhão de leads recebidos por 42 empresas americanas, mostrou que quem fez contato **em até uma hora** teve **sete vezes mais chance de qualificar o lead** do que quem respondeu uma hora depois, e mais de **60 vezes** mais chance do que quem demorou 24 horas ou mais. No mercado imobiliário o efeito é ainda mais forte, porque o mesmo cliente costuma pedir informação a várias imobiliárias ao mesmo tempo: quem responde primeiro, com opções concretas, ganha a visita.

Como a comissão de corretagem só é paga quando o negócio fecha, cada lead que esfria sem resposta é receita perdida para a imobiliária e para o corretor. O Sr. Agim ataca exatamente esse ponto: resposta imediata, qualificação consistente e acompanhamento que não depende da memória de ninguém.

2.3 A jornada do cliente: antes e depois

Momento	Sem o agente	Com o Sr. Agim
Sexta, 22h	Cliente envia mensagem pelo anúncio e não recebe resposta	Recebe resposta na hora, informa o CPF e diz o que procura
Sexta, 22h05	Pede informação a outras duas imobiliárias	Vê três apartamentos da base, com fotos, preço e R\$/m ² comparado ao bairro
Sexta, 22h10	—	Agenda a visita de sábado às 10h com o corretor da zona
Sábado, 9h	Corretor responde; o cliente já marcou visita com outra imobiliária	Corretor abre a agenda e lê o resumo do cliente antes da visita
Sábado, 12h	—	Pós-visita pergunta "o que achou?"; o motivo de uma recusa muda as próximas sugestões
Semana seguinte	Cliente some e ninguém retoma	Follow-up retoma de onde parou, com argumentos reais da base

3. Objetivos

3.1 Objetivo geral

Desenvolver um agente conversacional de IA Generativa que atenda leads de uma imobiliária de forma automática e humanizada, qualifique o interesse, consulte uma base de imóveis, agende visitas com o corretor adequado, faça o acompanhamento de quem para de responder e entregue resumos aos corretores.

3.2 Objetivos específicos

- Atender leads automaticamente pelo navegador e pelo Telegram, inclusive por áudio.
- Conversar de forma humanizada, reconhecendo o cliente que volta e mantendo o contexto entre sessões e canais.
- Identificar a intenção (compra, aluguel, investimento) e coletar região, quartos, orçamento e urgência; para investidores, o valor disponível e o retorno esperado.
- Consultar uma base simulada de imóveis com busca estruturada e semântica (RAG), sem oferecer o que não existe.
- Agendar visitas em linguagem natural, escolhendo o corretor pela região e pela carga de trabalho, e permitir que o cliente consulte, remarque e cancele as próprias visitas.
- **Captar imóveis de proprietários para venda e para aluguel:** cadastro guiado (tipo, quartos, banheiros, vagas, metragem, endereço), estimativa de valor a partir da base, avaliação agendada com o corretor da região, publicação do imóvel captado e cruzamento com os clientes interessados.
- Fazer follow-up automático, com limite de mensagens e respeito ao pedido de não ser contatado, e pós-visita que use a opinião do cliente nas próximas sugestões.
- Gerar resumos para os corretores e oferecer uma **Área do Corretor** (agenda, resultados, carteira, captações e desempenho) e um **Painel da Imobiliária** para o gestor.
- Organizar o código em **Clean Architecture com os princípios SOLID**, para que cada peça (LLM, banco, busca, voz, CRM) possa ser testada e trocada isoladamente.
- Proteger dados e credenciais: CPF validado e mascarado, segredos fora do código e do Git.
- Manter o projeto executável sem credenciais externas, com provedor simulado e testes automatizados.

4. Visão geral da solução

A solução foi dividida em módulos independentes. Três públicos usam o sistema por dois canais e, por trás, um único núcleo — serviços de aplicação e um grafo de agentes especializados — atende todos eles e grava tudo no mesmo banco de dados. Essa separação tornou o desenvolvimento mais organizado e permitiu testar cada parte sem depender das demais.

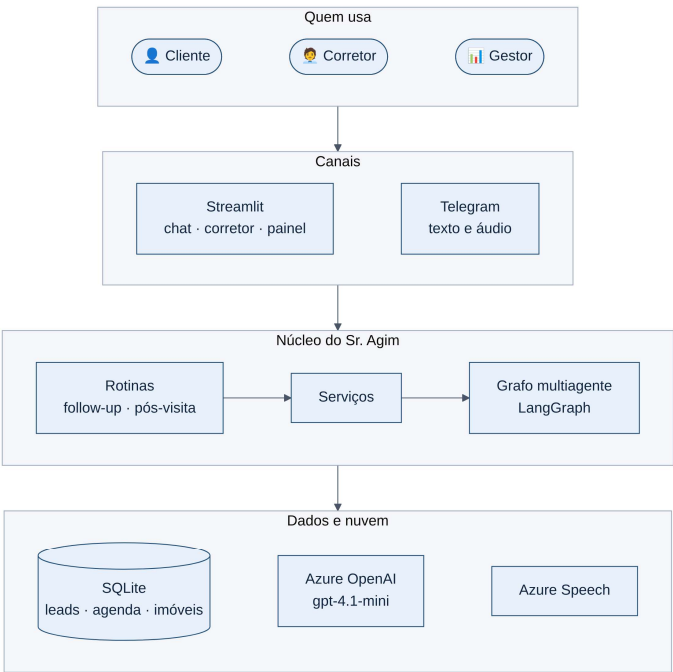


Figura 1 — Visão geral: quem usa, por onde entra e o que há por trás.

A divisão em vários agentes foi adequada ao problema porque o atendimento tem etapas muito diferentes. Identificar o cliente exige precisão absoluta; qualificar exige interpretar linguagem livre; buscar imóveis exige consultar dados; agendar exige regras de negócio; resumir exige boa redação. Cada etapa ganhou um agente próprio, com a técnica mais adequada para ela.

Público	O que ele quer	O que o Sr. Agim entrega
Cliente (compra ou aluguel)	Resposta rápida e opções que caibam no bolso	Atendimento imediato, imóveis da base com fotos, visita agendada, remarcar e cancelar pelo chat
Investidor	Saber se o imóvel rende mais que o dinheiro aplicado	Retorno estimado com FipeZAP, Selic e CDI e encaminhamento ao especialista
Proprietário	Saber quanto vale o imóvel e anunciá-lo	Cadastro rápido, faixa de valor estimada e avaliação presencial agendada
Corretor	Tempo para visitar e negociar	Agenda, resumo de cada cliente, carteira, sugestões, registro de resultados e o próprio funil
Gestor	Saber o que está acontecendo e onde agir	Funil, ranking, demanda × oferta, satisfação e itens que pedem atenção

4.1 Proposta de valor e indicadores de negócio

Um agente SDR só se justifica se melhorar números que a imobiliária já acompanha. Por isso, os eventos gravados a cada conversa alimentam indicadores que o gestor vê no Painel e o corretor vê na própria área. A tabela mostra o que pode ser medido com a POC e por que cada indicador importa para o negócio.

Indicador	Por que importa	Como o Sr. Agim mede
Tempo até a primeira resposta	É o fator que mais pesa na qualificação do lead	Resposta imediata em qualquer horário (evento mensagem_recebida → turno_processado)
Leads qualificados	Mostra quantos contatos têm interesse real	Temperatura frio / morno / quente calculada em código
Conversão lead → visita	Visita é o passo que mais aproxima do fechamento	Agendamentos confirmados sobre leads atendidos (Painel, funil)
Conversão visita → proposta → fechamento	Mede a qualidade da triagem e do corretor	Resultado registrado pelo corretor depois de cada visita
Comparecimento	Visita sem cliente é tempo perdido	Resultado nao_compareceu no pós-visita
Motivos de perda	Indica se o problema é preço, tamanho ou localização	Classificados no pós-visita (Preço, Localização, Tamanho, Estado, Condomínio, Outro)
Demanda × oferta por bairro	Mostra onde falta imóvel para captar	Buscas dos clientes cruzadas com a base (Painel)
Captações	Alimentam a oferta e a receita futura	Avaliações pedidas e imóveis captados por corretor
Satisfação	Mostra se o atendimento automático agrada	Nota de 1 a 5 no encerramento
Mensagens até agendar	Mede o esforço do cliente na conversa	Contagem de mensagens até a visita confirmada (Painel, métricas da IA)

Como comprovar o ganho numa imobiliária real

A POC não foi testada com clientes reais, então os ganhos ainda precisam ser medidos. A forma mais simples seria um **piloto de 60 a 90 dias**: metade dos anúncios direcionada ao Sr. Agim e metade ao atendimento atual, comparando tempo de primeira resposta, taxa de visita, taxa de fechamento e satisfação. Os indicadores acima já são registrados pelo sistema, então o piloto não exige desenvolvimento extra.

4.2 Regras de negócio implementadas

As decisões que afetam o cliente e o corretor estão em código Python, com valores explícitos e testados. A tabela reúne as principais regras e onde cada uma fica.

Regra	Como funciona	Onde está
Dados completos (compra/aluguel)	Região, quartos, faixa de preço (mínimo ou máximo) e urgência	qualifier_agent.py
Dados completos (investimento)	Valor disponível (ticket) e retorno esperado	qualifier_agent.py
Temperatura do lead	Pontos: intenção definida +1, dados completos +1, urgência imediata +1, quer agendar +2. Três ou mais = quente; um ou mais = morno; zero = frio	qualifier_agent.py
Quem pode agendar	Só lead morno ou quente que pediu visita	graph.py
Ordem da busca	Bairro pedido → bairros vizinhos (com pergunta ao cliente) → zona → outra região (sinalizada) → RAG	property_agent.py
Imóveis mostrados	No máximo três por vez; opções "próximas" aceitam até 40% acima do teto e um quarto a menos	property_agent.py

Regra	Como funciona	Onde está
Imóveis para investidor	Só venda, preço entre 50% e 105% do ticket, ordenados pelo retorno bruto	property_agent.py
Escolha do corretor	Corretor da zona com menos visitas ativas; investimento vai ao especialista; remarcação mantém o mesmo corretor	scheduler_agent.py
Conflito de agenda	Não marca visita a menos de uma hora de outra do mesmo corretor ou cliente	agenda_service.py, agenda_cliente_agent.py
Visita duplicada	Não cria outra visita para o mesmo CPF no mesmo horário	scheduler_agent.py
Follow-up	Lead sem resposta há 60 minutos e sem visita marcada; no máximo duas mensagens, ou três se ficou negócio pela metade; zera quando o cliente volta a falar	followup_agent.py, settings.toml
Opt-out	"Não quero mais contato" encerra o follow-up para aquele cliente	encerramento_agent.py
Pós-visita	Duas horas depois de uma visita confirmada (não vale para avaliação nem para quem não compareceu)	pos_visita_service.py
Recusa realimenta a busca	Preço ou condomínio → opções mais baratas; tamanho → maiores; localização → outro bairro	feedback_visita.py
Estimativa de valor	Faixa de $\pm 10\%$ (com metragem) ou $\pm 15\%$ (sem metragem) em torno da mediana dos comparáveis	avaliacao_imovel.py

4.3 Custo estimado de operação

Como o Python faz a maior parte do trabalho, o modelo é chamado poucas vezes por mensagem e recebe um contexto enxuto. A estimativa abaixo usa os preços públicos de referência do Azure (em dólar, sujeitos a alteração) e premissas conservadoras; **não é uma medição** e deve ser conferida na calculadora de preços do Azure antes de qualquer decisão.

Item	Valor de referência / premissa
Azure OpenAI gpt-4.1-mini	US\$ 0,40 por milhão de tokens de entrada e US\$ 1,60 por milhão de saída
Premissa por mensagem do cliente	Até duas chamadas ao modelo (extração + resposta), cerca de 5 mil tokens de entrada e 500 de saída no total
Custo por mensagem	≈ US\$ 0,003
Conversa típica (12 mensagens até agendar)	≈ US\$ 0,04 — menos de R\$ 0,25 por lead atendido
1.000 leads por mês	≈ US\$ 40 de modelo de linguagem
Voz (Azure Speech)	Cobrança por hora de áudio transcrito e por milhão de caracteres falados; o plano gratuito inclui 5 horas de transcrição e 0,5 milhão de caracteres por mês
Embeddings da busca	Sem custo: o modelo roda localmente

Mesmo com folga nas premissas, o custo do modelo por lead fica muito abaixo do valor de uma única visita gerada. O custo relevante de uma implantação real estaria na infraestrutura (banco gerenciado, hospedagem e integração com CRM) e não nas chamadas ao LLM.

5. Arquitetura e fluxo de execução

O código segue a **Clean Architecture**: cinco camadas em que as dependências apontam para dentro. O domínio não conhece banco, Azure, Telegram nem Streamlit; a infraestrutura implementa as interfaces do domínio. Todas as peças são montadas em um só lugar, o `src/container.py`.

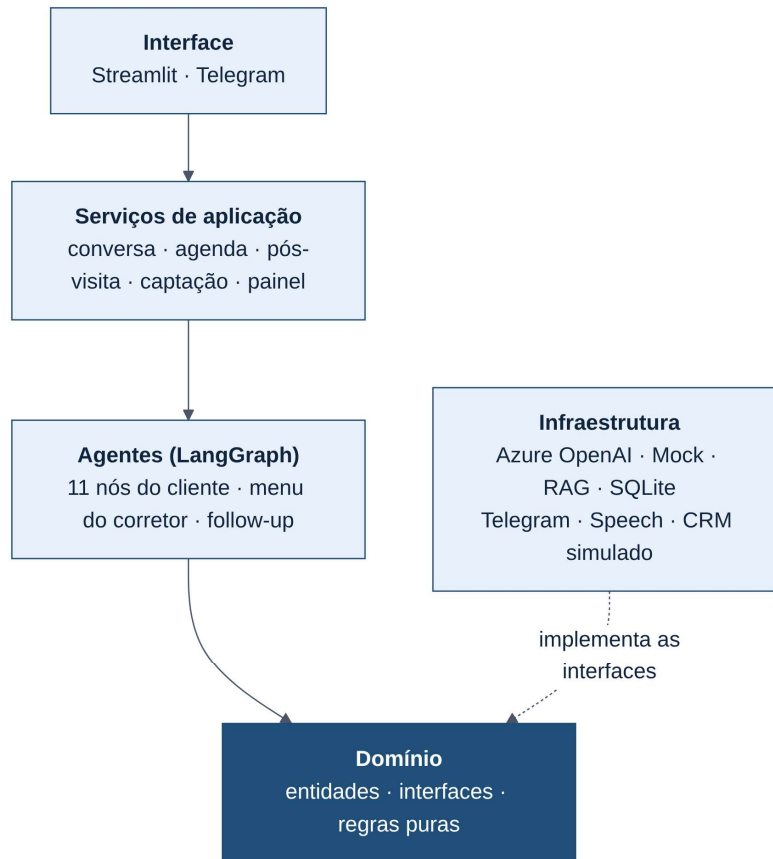


Figura 2 — Camadas da aplicação. A seta pontilhada é a inversão de dependência.

O atendimento ao cliente é um grafo LangGraph com 11 nós. Funções de roteamento em Python decidem quem age a seguir. Os nós de entrada (azul) mexem em dados sensíveis — CPF, agenda, cadastro — e são determinísticos; os nós de conversa (laranja) usam o LLM, sempre a partir de dados já calculados.

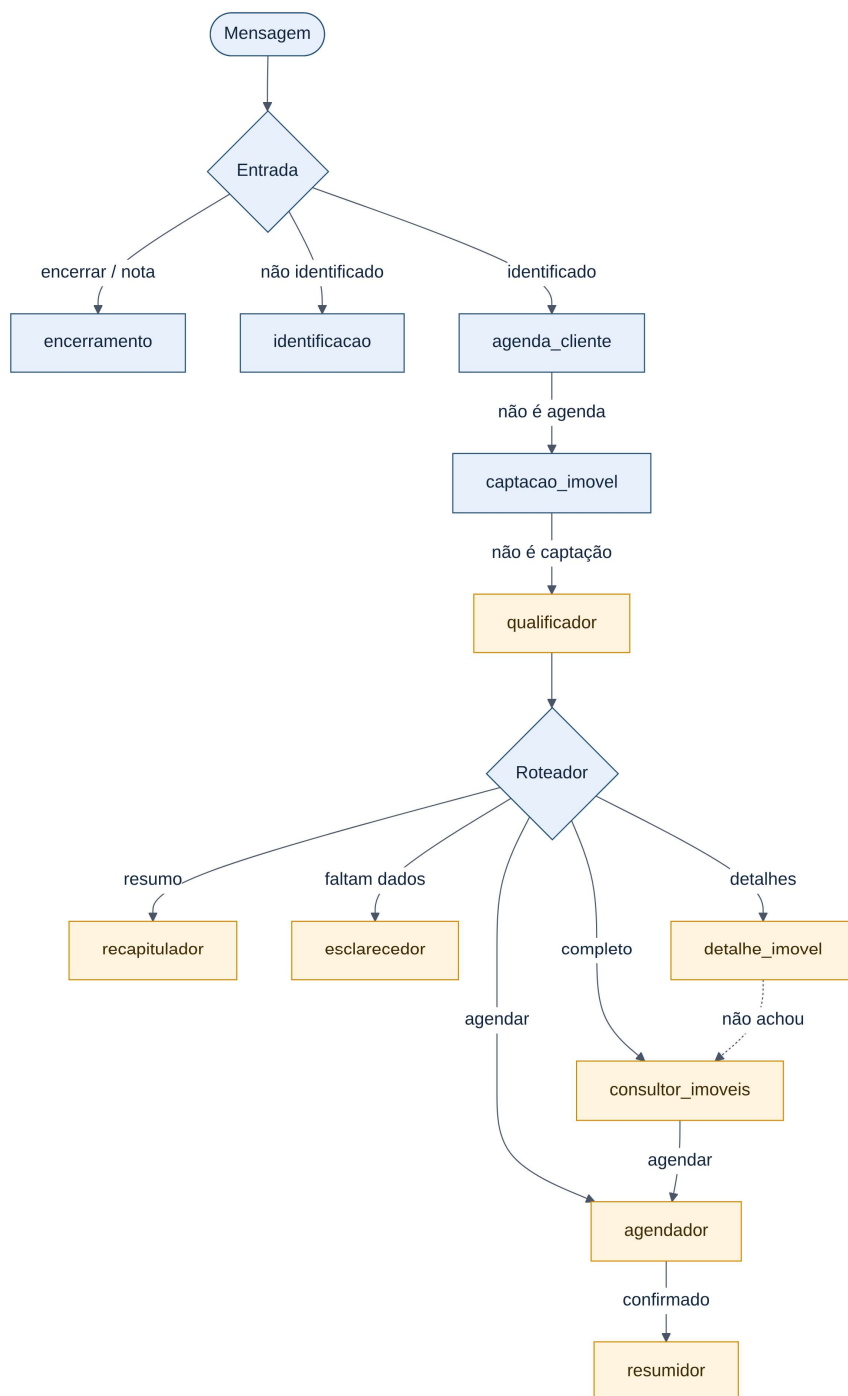


Figura 3 — Fluxo de execução do atendimento: o grafo multiagente do cliente.

5.1 Passo a passo do processamento

1. O cliente envia uma mensagem pelo chat web ou pelo Telegram. Se for áudio, o Azure Speech transcreve para texto.
2. O **ConversationService** carrega o **Lead** salvo (histórico e perfil) e monta o estado inicial do grafo.
3. O roteador de entrada decide o caminho: encerramento, identificação por CPF, agenda do próprio cliente ou captação de imóvel.
4. Se for uma busca, o qualificador extrai os dados da mensagem e o roteador escolhe o próximo agente: esclarecer o que falta, mostrar imóveis, detalhar um imóvel, agendar ou recapitular.
5. O consultor de imóveis busca na base (SQL e, se preciso, RAG) e o Python calcula R\$/m², rentabilidade e se o imóvel cabe no orçamento.

6. O agendador escolhe o corretor pela zona e pela carga, confere conflitos e duplicidade e registra a visita.
7. O LLM redige a resposta a partir desses fatos; o resumidor prepara o resumo para o corretor quando a visita é confirmada.
8. O serviço aplica o resultado no **Lead**, salva no SQLite, registra no CRM simulado e grava os eventos de observabilidade.
9. A interface mostra a resposta com listas, fotos e cartões; o corretor e o gestor veem os dados atualizados no Streamlit.
10. Em paralelo, as rotinas do bot verificam quem precisa de follow-up ou de pós-visita.

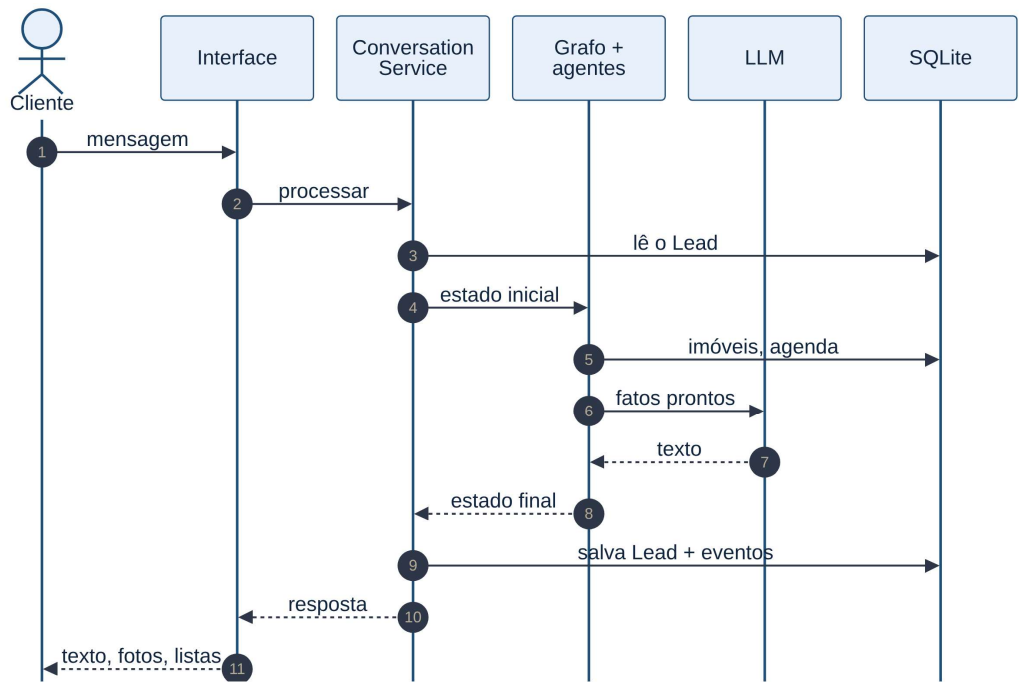


Figura 4 — Sequência de uma rodada de conversa.

5.2 Estado da conversa

Os agentes não conversam entre si diretamente: todos leem e escrevem num estado compartilhado, o **EstadoConversa** (um `TypedDict` em `src/agents/state.py`). O estado é montado a partir do **Lead** salvo no começo de cada mensagem e descartado no final, depois que o resultado é aplicado de volta ao **Lead**. Assim, a memória fica no banco e o grafo pode ser reiniciado a qualquer momento sem perder nada.

Grupo	Campos principais	Quem preenche
Entrada	lead_id, mensagem_usuario, historico_mensagens	ConversationService
Identificação	cliente_identificado, cliente_cpf, cliente_nome, lead_id_recuperado	Identificação
Perfil	intencao, faixa_preco_min/max, quartos_desejados, regioao_interesse, urgencia, ticket_investimento, expectativa_retorno, temperatura	Qualificador
Sinais de roteamento	quer_agendar, pediu_resumo, dados_completos, pediu_detalhes, nova_busca, ampliar_busca, imovel_interesse_id	Qualificador e agentes de entrada
Resultado	imoveis_sugeridos, resposta_agente	Consultor, detalhe, esclarecedor
Agenda	agendamento_data_hora, agendamento_status, corretor_id, corretor_nome, tipo_agendamento	Agendador e agenda do cliente

Grupo	Campos principais	Quem preenche
Pós-visita e captação	feedback_visita_novo, captacao, captacao_concluida	Agenda do cliente e captação
Encerramento	atendimento_encerrado, nota_atendimento, nao_contatar	Encerramento

5.3 Regras de roteamento do grafo

O caminho de cada mensagem é decidido por funções Python simples em `src/agents/graph.py`, e não pelo LLM. Isso torna o fluxo previsível e permite testar cada caminho isoladamente.

Ponto de decisão	Condição	Próximo nó
Entrada	Cliente pediu para encerrar ou está dando a nota	encerramento
Entrada	Cliente ainda não identificado	identificacao
Entrada	Cliente identificado	agenda_cliente
Depois da agenda do cliente	Mensagem era sobre a agenda dele (ver, cancelar, remarcar, opinar)	fim
Depois da agenda do cliente	Caso contrário	captacao_imovel
Depois da captação	Mensagem era do cadastro do imóvel	fim; senão qualificador
Depois da qualificação	Pediu "o que já conversamos?"	recapitulador
Depois da qualificação	Quer agendar e o lead está morno ou quente	agendador
Depois da qualificação	Pediu detalhes ou fotos de um imóvel	detalhe_imovel
Depois da qualificação	Faltam dados	esclarecedor
Depois da qualificação	Dados completos	consultor_imoveis
Depois dos imóveis	Cliente já quer agendar	agendador; senão fim
Depois do agendamento	Visita confirmada ou lead quente	resumidor; senão fim

5.4 Modelo de dados

As entidades ficam em `src/domain/entities.py` e não sabem como são gravadas. A persistência usa dois arquivos SQLite: `data/agente_sdr.db` (leads, agenda, corretores e eventos) e `data/imoveis.db` (base de imóveis, criada a partir de `data/imoveis.json` na primeira execução). Colunas novas são acrescentadas por migração automática, sem apagar dados.

Tabela	Colunas principais	Entidade
leads	id, dados_json (lead completo: perfil, histórico, buscas, captações, notas), ultima_interacao_em	Lead, PerfilLead
agendamentos	id, lead_id, tipo (visita, reunião, avaliação), status, data_hora, cliente, corretor, imóvel; cancelamento e remarcação (quem, motivo, quando, avisado); resultado da visita, feedback e valor_negociado	Agendamento
imoveis	id, titulo, tipo_negocio, tipo_imovel, zona, bairro, preco, condominio, quartos, suites, vagas, metragem, descricao, fotos, cadastrado_em, cadastrado_por	Imovel
corretores	id, nome, zonas_atuacao, email, telefone	Corretor
eventos	id, nome, dados_json, criado_em	Observabilidade

O **Lead** é gravado como JSON numa única coluna porque é lido e escrito inteiro a cada mensagem e muda com frequência durante o desenvolvimento. Os agendamentos, ao contrário, têm colunas próprias, porque são consultados por corretor, data e status pela agenda, pelo Painel e pelas rotinas automáticas.

5.5 Algoritmos principais

Busca de imóveis

1. Consulta SQL parametrizada com tipo de negócio, região (bairro ou zona), faixa de preço e quantidade mínima de quartos.
2. Se o bairro pedido não tiver resultado exato, procura opções próximas no mesmo bairro (até 40% acima do teto e um quarto a menos).
3. Se ainda faltar, oferece os bairros vizinhos (até quatro, de `data/bairros_sp.json`) e depois a zona inteira.
4. Como último recurso, a busca semântica (RAG) procura os oito imóveis mais parecidos com o pedido e aplica os mesmos filtros de negócio, preço e quartos.
5. O Python monta a lista final (no máximo três imóveis), calcula R\$/m² e compara com a média do bairro antes de o LLM redigir a resposta.

Estimativa de valor na captação

A função `estimar_valor` (`src/domain/avaliacao_imovel.py`) seleciona comparáveis do mesmo tipo de negócio, da mesma família de imóvel (apartamento, casa ou comercial) e com até um quarto de diferença, primeiro no bairro, depois nos vizinhos e por fim na zona. Em seguida:

- calcula valores candidatos: R\$/m² de mercado do bairro × metragem e mediana do R\$/m² dos comparáveis × metragem (sem metragem, usa a mediana dos preços);
- o centro da faixa é a mediana dos candidatos; a margem é de ±10% com metragem e ±15% sem metragem;
- a faixa final vai do menor entre o menor candidato e o centro menos a margem até o maior entre o maior candidato e o centro mais a margem;
- o resultado é arredondado para múltiplos de R\$ 50 (aluguel) ou R\$ 5 mil (venda).

Análise de investimento

A função `analisar_investimento` (`src/domain/investimento.py`) estima o aluguel pelo R\$/m² de locação do bairro × metragem (ou, sem esse dado, pelo rendimento médio de aluguel da cidade) e calcula:

```
rentabilidade_bruta_anual = aluguel_mensal * 12 / preço
rentabilidade_real       = (1 + bruta) / (1 + IPCA) - 1
poupança                 = 0,5% ao mês se Selic > 8,5%, senão 70% da Selic
comparação               = bruta comparada com CDI, Selic e poupança
```

Os indicadores vêm de `data/mercado_investimento.json`, com data de referência e fonte (FipeZAP, Copom, B3, IBGE e FGV), e são atualizados manualmente. A análise informa ainda se o retorno esperado pelo cliente está acima, dentro ou abaixo do mercado. Ela não desconta imposto de renda nem custos de manutenção e vacância, por isso deve ser lida como uma estimativa preliminar.

Escolha do corretor

- Investimento: corretor com a área "Investimentos" (especialista).
- Remarcação: mantém o corretor original.

- Demais casos: corretores cuja zona de atuação inclui a zona do imóvel (ou a região pedida); entre eles, o que tem **menos visitas ativas**, com o nome como desempate. Sem corretor na zona, vale qualquer corretor que não seja o especialista.
- Antes de gravar, confere conflito de horário (menos de uma hora) e se o cliente já tem visita no mesmo horário.

5.6 Clean Architecture e princípios SOLID

A organização do código foi pensada para que cada peça possa ser testada e trocada sem efeito cascata. A tabela resume as camadas; em seguida, cada princípio SOLID aparece com o arquivo, a classe e um trecho real do projeto. O detalhamento completo está em [docs/SOLID.md](#).

Camada	Pasta	Conhece	Não conhece
Domínio	src/domain/	Só Python	Banco, Azure, LangGraph, Streamlit
Agentes	src/agents/	Interfaces do domínio	Qual banco ou LLM está por trás
Serviços	src/services/	Domínio e agentes	Telas e canais
Infraestrutura	src/infrastructure/	Domínio (implementa as interfaces)	Agentes e telas
Interface	src/interface/, telegram_bot.py	Serviços	Detalhes de banco e de LLM

S — Responsabilidade única

Uma classe deve ter um único motivo para mudar. Cada agente cuida de uma etapa do atendimento e cada serviço de um caso de uso. As regras de negócio puras também ficam isoladas, cada uma em seu módulo de domínio.

Classe / módulo	Única responsabilidade
IdentificacaoAgent	Cadastrar ou reconhecer o cliente pelo CPF
QualificadorAgent	Extrair dados da mensagem e classificar o lead
AgendadorAgent	Escolher o corretor e registrar a visita
CaptacaoImovelAgent	Cadastro do imóvel do proprietário (venda ou aluguel)
ConversationService	Orquestrar uma rodada de conversa
AgendaService	Cancelar, remarcar e avisar cliente e corretor
src/domain/cpf.py	Validar o CPF pelo algoritmo oficial
src/domain/avaliacao_imovel.py	Estimar a faixa de valor do imóvel captado
src/domain/investimento.py	Calcular rentabilidade e comparar com Selic/CDI
Settings × fontes de configuração	Settings só guarda valores; quem lê TOML e .env são as fontes

O — Aberto/fechado

Aberto para extensão, fechado para modificação. Um agente novo é um nó novo no grafo — foi assim que entraram a captação, a agenda do cliente e o encerramento, sem alterar os agentes existentes. Uma fonte de configuração nova (por exemplo, Azure Key Vault) é só uma classe nova que siga o protocolo

FonteConfiguracao.

```
# src/agents/graph.py
grafo.add_node("encerramento", EncerramentoAgent(agenda_repository))
grafo.add_node("agenda_cliente", AgendaClienteAgent(agenda_repository, ...))
grafo.add_node("captacao_imovel", CaptacaoImovelAgent(...))

# src/infrastructure/configuracao.py
class FonteConfiguracao(Protocol):
    nome: str
    def ler(self) -> dict[str, Any]: ...
```

L — Substituição de Liskov

Uma implementação pode ocupar o lugar de outra sem que quem a usa perceba. Cada interface do domínio tem implementações intercambiáveis. A prova prática são os 240 testes, que rodam os mesmos agentes com o `MockLLMProvider` no lugar do Azure. Em tempo de execução, se os embeddings falharem, o `TfidfVectorSearch` assume no lugar do `EmbeddingVectorSearch`.

Interface	Implementações	Onde se escolhe
ILLMProvider	AzureOpenAIProvider, MockLLMProvider	llm.provider (settings.toml)
IPropertyRepository	SqlitePropertyRepository, JsonPropertyRepository	src/container.py
IVectorSearch	EmbeddingVectorSearch, TfidfVectorSearch	embeddings.provider
IEmbedder	SentenceTransformerEmbedder, AzureOpenAIEmbedder	embeddings.provider
FonteConfiguracao	ArquivoTomlFonte, AmbienteFonte	fontes_padrao() (config.py)

I — Segregação de interfaces

Ninguém deve depender de métodos que não usa. As interfaces de `src/domain/interfaces.py` são pequenas e focadas, e os repositórios são separados por assunto (leads, agenda, corretores, imóveis). O agente de captação, por exemplo, recebe só agenda, corretores, imóveis, bairros, mercado e CRM — não recebe o repositório de leads, porque não precisa dele.

Interface	Métodos	Quem usa
IVectorSearch	buscar_similares	Consultor de imóveis
INotifier	enviar	Avisos pelo Telegram
IVoiceService	texto_para_fala, fala_para_texto	Bot e chat web
IMarketDataRepository	obter_indicadores	Investimento e captação
ICRM	registrar_lead, registrar_resumo, registrar_agendamento	Conversa e agendador
IObservador (Protocol)	registrar_evento	Conversa e rotinas

D — Inversão de dependência

Módulos de alto nível não dependem de detalhes; os dois dependem de abstrações. Agentes e serviços recebem interfaces no construtor e nunca criam suas dependências. Quem decide as implementações concretas é um único lugar, o composition root `src/container.py`, com fábricas como `criar_llm_provider`, `criar_busca_semantica` e `criar_servico_de_voz`.

```
# src/services/conversation_service.py
class ConversationService:
    def __init__(self, grafo_compilado, lead_repository: ILeadRepository,
                  crm: ICRM, observador: IObservador, resumidor=None): ...

# src/container.py (composition root)
llm_provider = criar_llm_provider(settings)
busca_semantica = criar_busca_semantica(repositorio_imoveis, mapa_bairros)
agenda_repository = SqliteAgendaRepository(caminho_db=settings.database_path)
grafo = construir_grafo_sdr(llm_provider=llm_provider, ...)
```

Na prática: trocar o provedor de LLM

1. Criar uma classe nova que herda de `ILLMProvider` (por exemplo, `GeminiProvider`) e implementa `gerar_resposta` e `extrair_dados_estruturados`.
2. Acrescentar uma linha em `criar_llm_provider` para escolhê-la quando `llm.provider = "gemini"`.
3. Nenhum agente, serviço ou tela muda, e os testes continuam valendo. O mesmo vale para trocar o SQLite por Azure SQL ou o CRM simulado por um real.

Onde ainda dá para melhorar

`MockCRM()` usa um caminho fixo no container e `criar_busca_semantica` lê a configuração global — o ideal seria recebê-los do container, como já acontece com o LLM. Alguns arquivos cresceram além do ideal (`paginas/corretor.py` e `qualifier_agent.py`, com cerca de 500 linhas cada) e poderiam ser divididos. São melhorias localizadas, que não exigem mudar a arquitetura.

5.7 Tecnologias

Camada	Tecnologia
Orquestração de agentes	LangGraph 0.2 + LangChain 0.3 (Python 3.11+)
Modelo de linguagem	Azure OpenAI gpt-4.1-mini (East US), com provedor simulado para testes e uso sem chave
Busca de imóveis (RAG)	SQL parametrizado + embeddings locais sentence-transformers (TF-IDF como reserva)
Voz	Azure Speech via REST (fala → texto e texto → fala, voz neural pt-BR)
Interfaces	Streamlit (chat, Área do Corretor, Painel) e bot do Telegram
Dados	SQLite (leads, agenda, eventos, imóveis) + JSON de bairros, mercado e corretores
Qualidade e entrega	pytest (240 testes), Docker + docker-compose, configuração em camadas

Versões fixadas em `requirements.txt`, para que o ambiente de quem avalia seja igual ao de desenvolvimento:

Pacote	Versão	Uso no projeto
langgraph	0.2.60	Grafo de agentes, estado e roteamento
langchain / langchain-core / langchain-openai	0.3.13 / 0.3.28 / 0.2.14	Integração com o Azure OpenAI e estruturas de mensagem
openai	1.58.1	Cliente do Azure OpenAI (chat e saída JSON)
sentence-transformers	≥ 3.3	Embeddings locais para o RAG

Pacote	Versão	Uso no projeto
scikit-learn / numpy	1.5.2 / ≥ 2.0	TF-IDF de reserva e similaridade de cosseno
streamlit / plotly / pandas	1.40.2 / 5.24.1 / 2.2.3	Interface web, gráficos do Painel e tabelas
python-telegram-bot	21.9	Canal Telegram (polling, fotos e áudio)
httpx	0.28.1	Chamadas REST ao Azure Speech e testes com rede simulada
pydantic / python-dotenv	2.10.4 / 1.0.1	Validação de dados e leitura do .env
pytest	8.3.4	Testes automatizados

Parâmetros de IA por agente

Cada chamada ao modelo usa a temperatura adequada à tarefa: zero quando a saída precisa ser exata e mais alta quando o texto precisa soar natural.

Uso	Temperatura	Motivo
Extração de dados (saída JSON)	0	Precisa ser exata e repetível; usa <code>response_format = json_object</code>
Gestão de agenda pelo corretor	0	Interpreta comandos; não pode variar
Resumo para o corretor	0,2	Fiel aos fatos, sem enfeite
Recapitulação e consultas de agenda	0,3	Objetiva, mas em linguagem natural
Apresentação e detalhe de imóveis	0,4	Natural, com os fatos já montados
Perguntas do esclarecedor	0,5	Variar a forma de perguntar para não soar robótico
Follow-up e saudação	0,6	Mensagem calorosa e persuasiva

5.8 Configuração e segurança

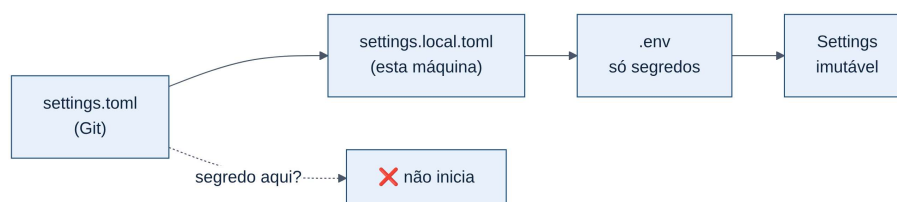


Figura 5 — De onde vem cada configuração. O último vence; segredo fora do .env impede a subida.

- Configuração em três camadas: `config/settings.toml` (vai para o Git), `config/settings.local.toml` (só da máquina, por exemplo o modo de demonstração com tempos curtos) e `.env` (somente chaves e tokens). Qualquer valor pode ser sobrescrito por variável de ambiente, como no Docker.
- Cada campo de **Settings** declara se é segredo. Um segredo encontrado em arquivo TOML faz o app **recusar a inicialização**, e segredos não aparecem em logs.
- O CPF é validado pelo algoritmo oficial e vai **mascarado** em tudo o que é enviado ao LLM.
- A interface mostra se o LLM ativo é o Azure (🌀) ou o simulado (🌀) e se a voz está ligada (🔊), e o nome do agente é configurável (`[agente] nome`).

5.9 Observabilidade

Cada mensagem e cada ação relevante geram um **evento estruturado**, gravado no SQLite e no log: `mensagem_recebida`, `turno_processado`, `agendamento_cancelado_pelo_cliente`, `agendamento_cancelado_pelo_corretor`, `resultado_visita_registrado`, `captacao_resultado`, `imovel_cadastrado`, `followup_disparado` e `pos_visita_enviado`, entre outros. O Painel da Imobiliária lê esses eventos direto do banco para montar os indicadores, e eles servem também para auditoria e depuração.

Evento	Quando é gravado
<code>mensagem_recebida / turno_processado</code>	A cada mensagem do cliente e ao fim do processamento
<code>agendamento_cancelado_pelo_cliente / ..._remarcado_pelo_cliente</code>	Cliente altera a própria visita pelo chat
<code>agendamento_cancelado_pelo_corretor / ..._remarcado_pelo_corretor</code>	Corretor altera a visita na Área do Corretor
<code>corretor_avisado / cliente_avisado_pelo_corretor</code>	Aviso entregue ao outro lado da agenda
<code>resultado_visita_registrado</code>	Corretor registra o resultado (gostou, proposta, fechado, não gostou, não compareceu)
<code>pos_visita_enviado</code>	Mensagem "o que achou?" enviada ao cliente
<code>captacao_resultado</code>	Corretor conclui a avaliação de um imóvel captado
<code>imovel_cadastrado / match_avisado</code>	Imóvel novo publicado e clientes compatíveis avisados
<code>followup_disparado</code>	Mensagem de retomada enviada a um lead parado

6. Soluções implementadas por módulo

Sobre as capturas de tela

As telas desta seção foram geradas com o sistema real, rodando com **dados de demonstração** e o **provedor de LLM simulado** (sem chave). Com o Azure OpenAI a redação fica mais natural; os fatos — imóveis, preços, horários, corretor — são os mesmos, porque vêm do Python. Nomes e CPFs são fictícios e as fotos dos imóveis são ilustrativas.

6.1 Atendimento ao cliente

O atendimento começa pela identificação: o cliente informa CPF e nome, validados em código pelo algoritmo oficial. Quem já falou antes é reconhecido e recupera o histórico. Depois, o qualificador entende a intenção e o esclarecedor pergunta só o que falta. A busca começa no bairro pedido, passa aos vizinhos e só então vai para a zona.

- Identificação determinística por CPF, sem passar pelo LLM; CPF mascarado em tudo o que vai para o modelo.
- Qualificação com temperatura do lead (frio, morno, quente) e urgência classificada em código.
- Lista curta de imóveis montada em código, fotos sob demanda e ficha com R\$/m² comparado ao bairro (FipeZAP).
- Agendamento em linguagem natural ("sexta às 10h"), com corretor por zona e carga e sem visita duplicada.
- Cliente consulta, remarca e cancela as próprias visitas pelo chat; o corretor é avisado na Área do Corretor.
- Quando o corretor cancela ou remarca, o cliente é avisado na hora, no chat e no Telegram.

 Prazer em te conhecer, Gabriela! Cadastro concluído (348.552.316-06). Me conta: você está buscando comprar, alugar ou investir em um imóvel?

 Quero comprar um apartamento de 2 quartos na Mooca até 600 mil

 Ótimo! E para quando você precisa do imóvel — é algo urgente ou sem pressa?

 preciso me mudar em uns 2 meses

 Separei estas opções para você:

1. Apartamento · 2 quartos · 52 m² · 1 vaga · Mooca
Valor de venda: R\$ 365.000,00 · Condomínio: R\$ 680,00
2. Apartamento · 2 quartos · 55 m² · 1 vaga · Mooca
Valor de venda: R\$ 495.000,00 · Condomínio: R\$ 720,00
3. Apartamento · 2 quartos · 64 m² · 1 vaga · Mooca
Valor de venda: R\$ 595.000,00 · Condomínio: R\$ 830,00

Quer ver as fotos ou mais detalhes de algum? É só me dizer o número (ex.: "fotos do 1").
Se nenhuma dessas te agradar, também tenho opções nos bairros vizinhos (Belém, Ipiranga, Tatuapé, Vila Prudente). Quer que eu te mostre as opções nos bairros vizinhos?

Figura 6 — Qualificação e lista de imóveis montada a partir da base.

- Localização: Mooca, Zona Leste
- 2 quarto(s), 52 m²
- Preço: R\$ 365.000,00
- Descrição: 2 dormitórios, 1 vaga, cozinha planejada, prédio com elevador. A 8 min a pé da estação Bresser-Mooca do metrô.
- Fotos: 4 imagens (exibidas abaixo da mensagem)
- Bairros vizinhos: Belém, Brás, Cambuci, Ipiranga, Tatuapé, Vila Prudente, Água Rasa
- Preço por m²: R\$ 7.019,23 — 24,52% abaixo da média do bairro Mooca (R\$ 9.300,00/m², Índice FipeZAP agosto/2026)
- Orçamento do lead: R\$ 600.000,00 — o imóvel cabe no orçamento

Quer agendar uma visita a este imóvel?

 Fotos — Apartamento reformado perto do Juventus (Mooca)



Fachada · imagem ilustrativa



Sala de estar · imagem ilustrativa



Cozinha · imagem ilustrativa



Quarto · imagem ilustrativa

Figura 7 — "Fotos do 1°": ficha do imóvel com R\$/m² comparado à média do bairro e galeria.



Que bom te ver de novo, Mariana! Já recuperei o nosso histórico de conversa. Da última vez falamos sobre compra de imóvel, na região Mooca, 2 quarto(s), até R\$ 600.000.



Você tem este agendamento comigo:

1. Visita dia 09/10 às 10h — Apartamento reformado perto do Juventus
Corretor(a): Beatriz Lima · (11) 98888-0006 ·  confirmada

Quer continuar de onde paramos ou prefere buscar outra coisa?

Figura 8 — Cliente recorrente reconhecido pelo CPF, com a agenda dele.

 526.018.159-06



Que bom te ver de novo, Mariana! Já recuperei o nosso histórico de conversa. Da última vez falamos sobre compra de imóvel, na região Mooca, 2 quarto(s), até R\$ 600.000.

 Você tem este agendamento comigo:

1. Visita dia 09/10 às 10h — Apartamento reformado perto do Juventus
Corretor(a): Beatriz Lima · (11) 98888-0006 ·  confirmada

Quer continuar de onde paramos ou prefere buscar outra coisa?

 quero remarcar a visita para sábado às 10h



Prontinho! Remarquei sua visita com o(a) corretor(a) Beatriz Lima: de dia 09/10 às 10h para dia 10/10 às 10h. Já avisei o corretor do novo horário. Posso ajudar em mais alguma coisa?

 sim



Sua visita com o(a) corretor(a) Beatriz Lima já está marcada para dia 10/10 às 10h  Posso te ajudar com mais alguma coisa?



Figura 9 — Remarcação pelo chat, com aviso ao corretor e sem duplicar a visita.

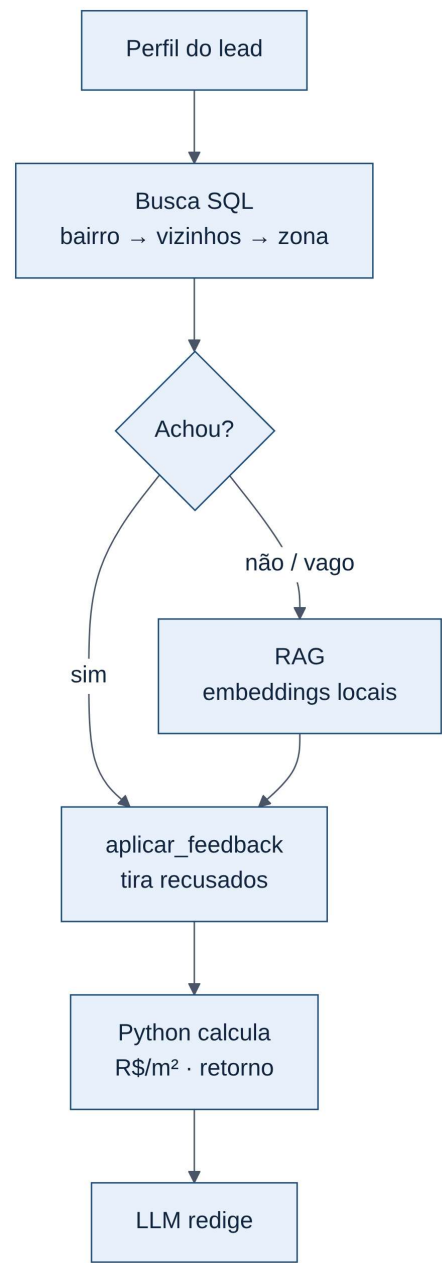


Figura 10 — Busca de imóveis: SQL primeiro, RAG como rede de segurança.

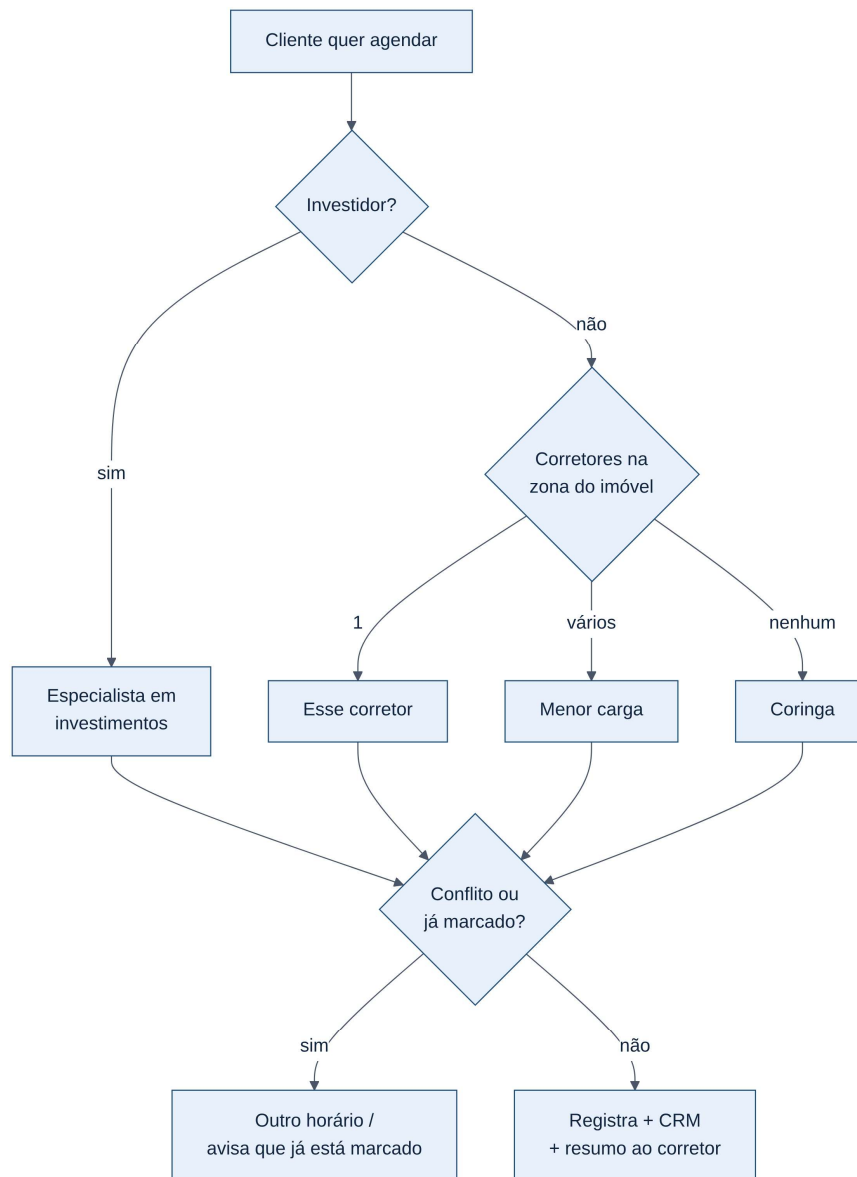
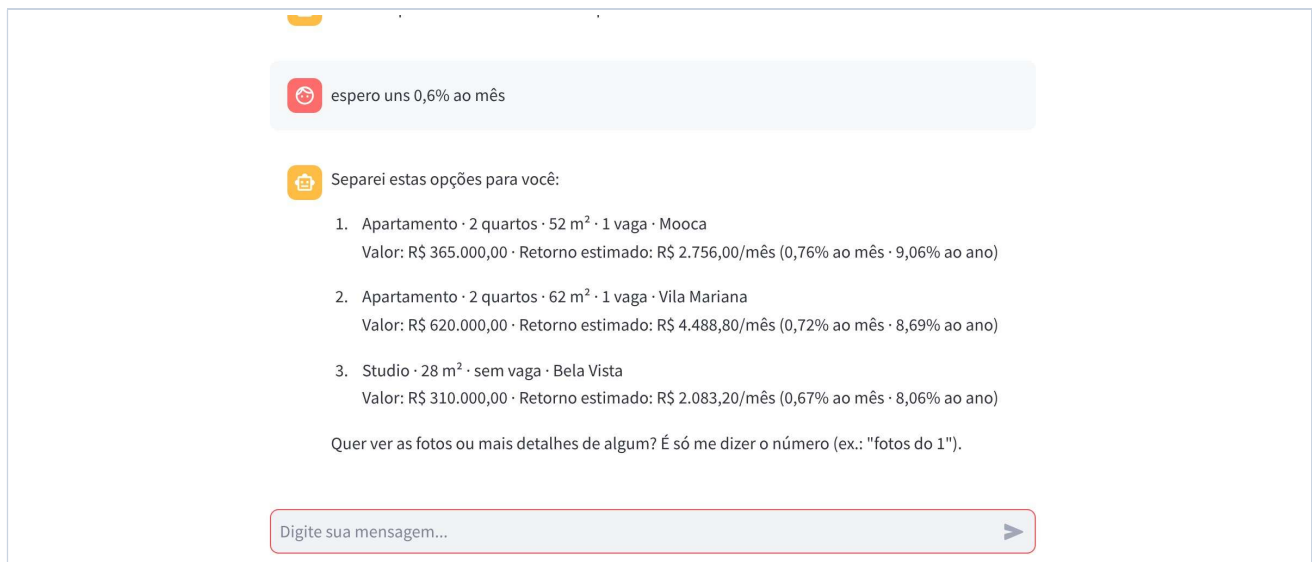


Figura 11 — Escolha do corretor: zona, carga, especialista e verificação de conflito.

6.2 Investimento

No cenário de investimento, o agente pergunta o valor disponível e o retorno esperado e compara os imóveis pela rentabilidade estimada, usando dados de mercado mantidos em `data/mercado_investimento.json` (FipeZAP, Selic, CDI e IPCA). A reunião é direcionada ao corretor especialista em investimentos.



 espero uns 0,6% ao mês

 Separei estas opções para você:

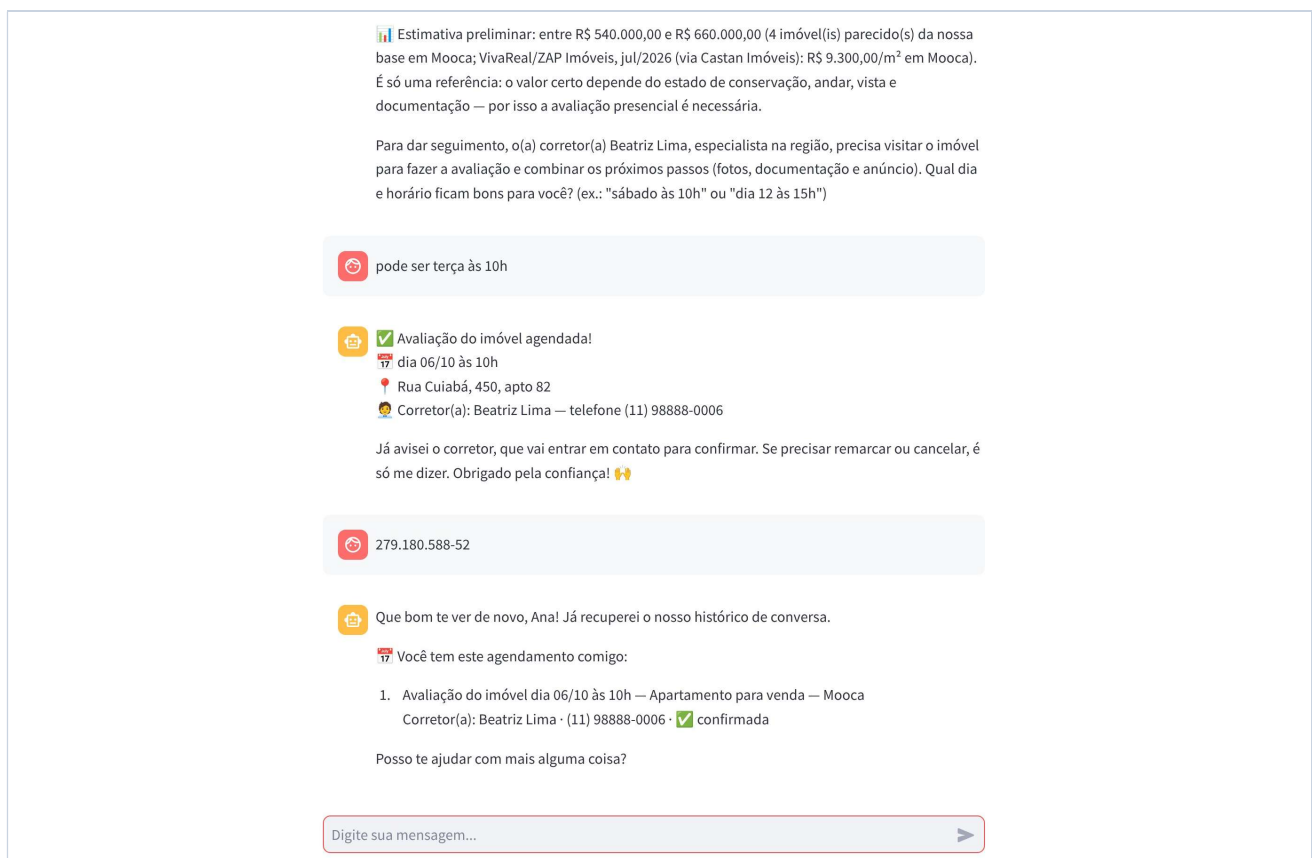
1. Apartamento · 2 quartos · 52 m² · 1 vaga · Mooca
Valor: R\$ 365.000,00 · Retorno estimado: R\$ 2.756,00/mês (0,76% ao mês · 9,06% ao ano)
2. Apartamento · 2 quartos · 62 m² · 1 vaga · Vila Mariana
Valor: R\$ 620.000,00 · Retorno estimado: R\$ 4.488,80/mês (0,72% ao mês · 8,69% ao ano)
3. Studio · 28 m² · sem vaga · Bela Vista
Valor: R\$ 310.000,00 · Retorno estimado: R\$ 2.083,20/mês (0,67% ao mês · 8,06% ao ano)

Quer ver as fotos ou mais detalhes de algum? É só me dizer o número (ex.: "fotos do 1").

Figura 12 — Investidor: imóveis comparados por retorno mensal e anual estimado.

6.3 Captação de imóveis

O proprietário que quer vender ou alugar o próprio imóvel passa por um cadastro guiado. A função de domínio `estimar_valor` calcula uma faixa de preço com imóveis comparáveis (bairro, vizinhos, zona) e o R\$/m² do bairro, e a avaliação presencial é agendada com o corretor da região. Quando o corretor marca como "captado", o imóvel é publicado na base e cruzado com os clientes interessados.



 Estimativa preliminar: entre R\$ 540.000,00 e R\$ 660.000,00 (4 imóvel(is) parecido(s) da nossa base em Mooca; VivaReal/ZAP Imóveis, jul/2026 (via Castan Imóveis): R\$ 9.300,00/m² em Mooca). É só uma referência: o valor certo depende do estado de conservação, andar, vista e documentação — por isso a avaliação presencial é necessária.

Para dar seguimento, o(a) corretor(a) Beatriz Lima, especialista na região, precisa visitar o imóvel para fazer a avaliação e combinar os próximos passos (fotos, documentação e anúncio). Qual dia e horário ficam bons para você? (ex.: "sábado às 10h" ou "dia 12 às 15h")

 pode ser terça às 10h

  Avaliação do imóvel agendada!
 dia 06/10 às 10h
 Rua Cuiabá, 450, apto 82
 Corretor(a): Beatriz Lima — telefone (11) 98888-0006

Já avisei o corretor, que vai entrar em contato para confirmar. Se precisar remarcar ou cancelar, é só me dizer. Obrigado pela confiança! 🙌

 279.180.588-52

 Que bom te ver de novo, Ana! Já recuperei o nosso histórico de conversa.

 Você tem este agendamento comigo:

1. Avaliação do imóvel dia 06/10 às 10h — Apartamento para venda — Mooca
Corretor(a): Beatriz Lima · (11) 98888-0006 ·  confirmada

Posso te ajudar com mais alguma coisa?

Figura 13 — Captação: estimativa de valor e avaliação agendada com o corretor da zona.

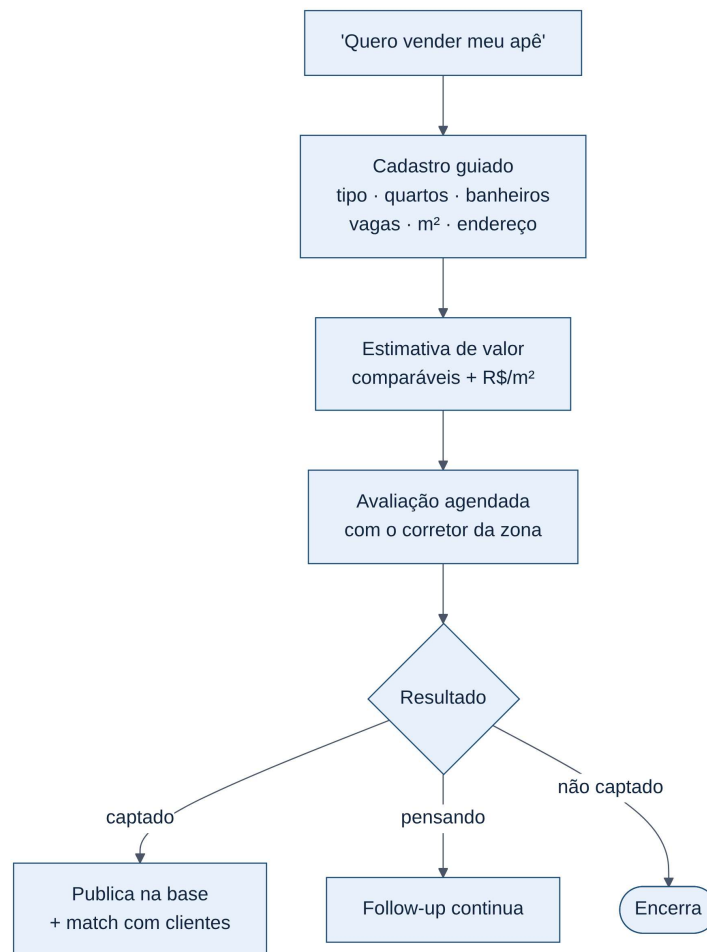


Figura 14 — Do "quero vender meu apê" à publicação do imóvel e ao match com clientes.

6.4 Follow-up, pós-visita e encerramento

As rotinas automáticas rodam dentro do bot do Telegram. O follow-up procura quem parou de responder e não tem visita marcada: no máximo duas mensagens, ou três quando ficou um negócio pela metade, com argumentos reais da base. O pós-visita pergunta "o que achou?" algumas horas depois da visita, e o motivo de uma recusa (preço, tamanho, localização) muda as próximas sugestões. O cliente pode encerrar o atendimento, dar uma nota de 1 a 5 e pedir para não ser mais contatado.

 pode encerrar o atendimento, obrigado

 Atendimento encerrado ✔️ Foi um prazer te ajudar, Lucas!
 Ficou registrado o seu interesse: aluguel de imóvel, na região Santana, 3 quarto(s), até R\$ 5.000.
Seu histórico fica salvo — quando quiser voltar, é só mandar uma mensagem que eu continuo de onde paramos.
Antes de ir: de 1 a 5, que nota você dá para este atendimento? (opcional)

 5

 Obrigado pela avaliação! ❤️ Quando precisar, é só mandar uma mensagem por aqui.

✔️ Atendimento encerrado. Quando quiser, é só mandar uma mensagem para continuar de onde parou.

Figura 15 — Encerramento: resumo do que ficou combinado e nota de satisfação.

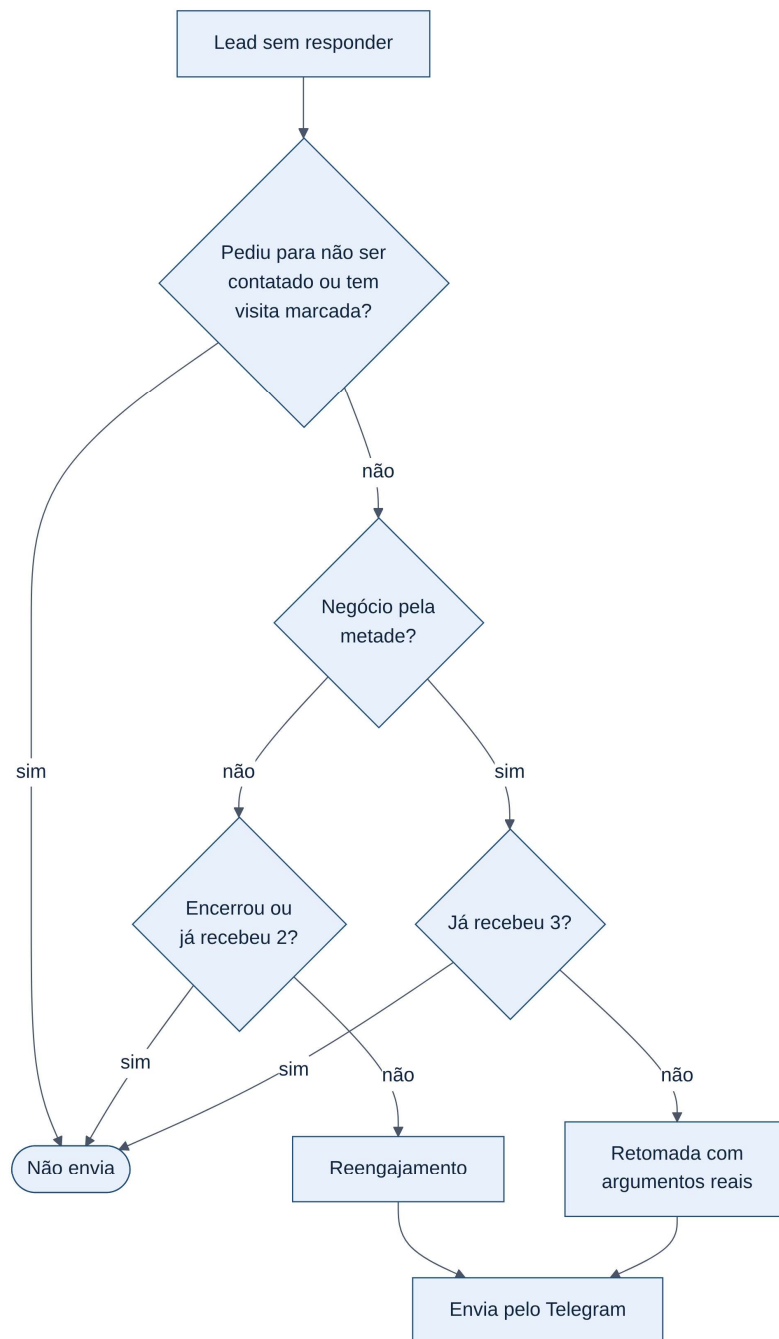


Figura 16 — Quem recebe follow-up e com qual tipo de mensagem.

6.5 Área do Corretor

A Área do Corretor reúne, em sete abas, tudo o que o corretor precisa: agenda com o resumo de cada cliente, registro do resultado das visitas, clientes da área que ainda não visitaram, imóveis da área, imóveis novos que combinam com clientes, captações e o próprio desempenho. Um assistente com menu de nove opções permite fazer o mesmo em linguagem natural.

#	Opções do assistente do corretor ("O que deseja fazer?")
1	Ver minha agenda
2	Registrar resultado de visita / negociação (proposta, fechado)
3	Clientes da minha área sem visita

#	Opções do assistente do corretor ("O que deseja fazer?")
4	Imóveis para sugerir aos clientes agendados
5	Imóveis novos × clientes interessados
6	Meu desempenho (funil de vendas)
7	Remarcar ou cancelar um agendamento
8	Captação de imóveis (avaliações de imóveis de clientes)
9	Imóveis da minha área (todos os cadastrados)

Avisos de clientes:

 Nova captação: Ana Beatriz Costa quer anunciar um imóvel — Apartamento para venda — Mooca. Avaliação dia 06/10 às 10h. Apartamento · 2 quarto(s) · 1 banheiro(s) · 1 vaga(s) · 65 m² · Rua Cuiabá, 450, apto 82 — Mooca (à venda)

 Mariana Alves remarcou a visita (Apartamento reformado perto do Juventus) do dia 09/10 às 10h para dia 10/10 às 10h.

Figura 17 — Avisos gerados quando o cliente altera a agenda ou pede uma avaliação.

 Agenda
  Pós-visita
  Clientes
  Imóveis da área
  Imóveis novos
  Captações
  Desempenho

Agendamentos ativos

3

 Meus agendamentos (3)

06/10/2026 às 10h00 · Avaliação do imóvel com Ana Beatriz Costa — Apartamento para venda — Mooca
status: *confirmado*

 Cancelar

 Ficha do imóvel (captação)

 Resumo do cliente

09/10/2026 às 15h00 · Visita com Gabriela Santos — Apartamento reformado perto do Juventus
status: *confirmado*

 Cancelar

 Resumo do cliente

10/10/2026 às 10h00 · Visita com Mariana Alves — Apartamento reformado perto do Juventus
status: *confirmado*
 Remarcado pelo cliente — antes: dia 09/10 às 10h

 Cancelar

 Resumo do cliente

Figura 18 — Agenda com ficha da captação, resumo do cliente e cancelamento com confirmação.

17 Agenda Pós-visita Clientes Imóveis da área Imóveis novos Captações Desempenho

Resultado das visitas

Registre como foi cada visita: alimenta o seu funil de vendas e, quando o cliente não gosta, o motivo volta para o Sr. Agim, que passa a sugerir opções melhores (ex.: mais baratas).

Negociações em andamento

Quando a proposta for aceita, marque 🏆 Negócio fechado e informe o valor — entra na hora no seu desempenho.

Juliana Rocha · Visita 03/10/2026 às 08h54 — Casa em condomínio fechado

Situação atual: 📝 Fez proposta · R\$ 820.000,00

Resultado

🏆 Negócio fechado

Motivo (se não gostou / não fechou)

Preço

Valor da proposta / do negócio (R\$)

820000.00

Observação (opcional)

Salvar

✅ Já registrados

- Ricardo Gomes · 01/10/2026 às 08h54: 🏆 Negócio fechado — R\$ 2.700,00

Figura 19 — Pós-visita: o corretor registra o resultado e o valor da proposta ou do negócio.

17 Agenda Pós-visita Clientes Imóveis da área Imóveis novos Captações Desempenho

🔍 Clientes da sua área sem visita agendada (1)

Leads que procuraram imóveis em Zona Oeste e ainda não marcaram visita/reunião. Os mais quentes e recentes aparecem primeiro — boa lista para ligar hoje.

Sofia Martins · 🌟 morno · canal web · última conversa 05/10/2026 10h54 (UTC)

Procura: compra de imóvel, na região Pinheiros, 2 quarto(s), até R\$ 1.000.000

Situação: Parou de responder

Imóvel de interesse: Apartamento 3 dormitórios para família (Pinheiros)

📝 Resumo do cliente

💡 Clientes agendados — imóveis para sugerir (1)

Além do imóvel escolhido pelo cliente, opções parecidas (mesmo bairro → vizinhos → mesma região, preço e quartos próximos; para investidor, maior retorno) para levar na visita. 🗨️ = o cliente ainda não viu no chat.

Carlos Menezes · Visita em 02/10/2026 às 08h54 · aluguel · status: confirmado

★ Escolhido pelo cliente: IM028 — Apartamento para alugar em Pinheiros

Apartamento · 2 quartos · 68 m² · 1 vaga · Pinheiros

Aluguel: R\$ 6.500,00/mês · Condomínio: R\$ 880,00

Não encontrei imóveis parecidos na base para sugerir.

Figura 20 — Clientes da área sem visita e imóveis extras para sugerir aos agendados.

Agenda
 Pós-visita
 Clientes
 Imóveis da área
 Imóveis novos
 Captações
 Desempenho

Captação de imóveis

Clientes que querem vender ou alugar o próprio imóvel: o Sr. Agim fez o cadastro, estimou o valor e agendou a avaliação. Depois da visita, registre o resultado — captado entra na base na hora, com o match de clientes interessados.

Ana Beatriz Costa · avaliação 06/10/2026 às 10h00 · venda

Apartamento · 2 quarto(s) · 1 banheiro(s) · 1 vaga(s) · 65 m² · Rua Cuiabá, 450, apto 82 — Mooca (à venda)

Estimativa preliminar: entre R\$ 540.000,00 e R\$ 660.000,00 (4 imóvel(is) parecido(s) da nossa base em Mooca; VivaReal/ZAP Imóveis, jul/2026 (via Castan Imóveis): R\$ 9.300,00/m² em Mooca)

Resultado

Valor de anúncio (R\$)

Motivo (se não captado)

Captado — public... ▼

0.00

— +

Salvar

Figura 21 — Captações aguardando avaliação; "captado" publica o imóvel na base.

Agenda
 Pós-visita
 Clientes
 Imóveis da área
 Imóveis novos
 Captações
 Desempenho

Visitas realizadas

1

Visita → proposta

100%

Negócios fechados ⓘ

1

Comparecimento ⓘ

100%

Funil de vendas



Por que não fecharam

Sem perdas registradas.

Agenda

Cancelados pelo cliente: 0 · por você: 0

Remarcações: 0

Valor fechado: R\$ 3.800,00 · Propostas em aberto: 0

Figura 22 — Desempenho: funil individual, taxas, valor fechado e motivos de perda.

Olá! Que bom te ver por aqui 🌟 Você não tem nenhum agendamento para esse período. 🗓️

O que deseja fazer, Camila? Escolha um número ou me diga com suas palavras:

1. 📅 Ver minha agenda
2. 📝 Registrar resultado de visita / negociação (proposta, fechado)
3. 🔍 Clientes da minha área sem visita
4. 💡 Imóveis para sugerir aos clientes agendados
5. 🏠 Imóveis novos × clientes interessados
6. 📊 Meu desempenho (funil de vendas)
7. 🗓️ Remarcar ou cancelar um agendamento
8. 🏠 Captação de imóveis (avaliações de imóveis de clientes)
9. 🏠 Imóveis da minha área (todos os cadastrados)

6

Seu desempenho

Leads da área: 2 → Visitas agendadas: 1 → Visitas realizadas: 1 → Propostas: 1 → Negócios fechados: 1

Visita → proposta: 100% · Fechamento: 100% · Comparecimento: 100%

Cancelamentos: 0 pelo cliente, 0 por você · Remarcações: 0

Negócios fechados: R\$ 3.800,00 · Propostas em aberto: 0

Os gráficos estão na aba 📊 **Desempenho**.

Digite **menu** para ver as opções de novo.

Figura 23 — Assistente do corretor: menu "O que deseja fazer?" e resposta a partir dos dados.

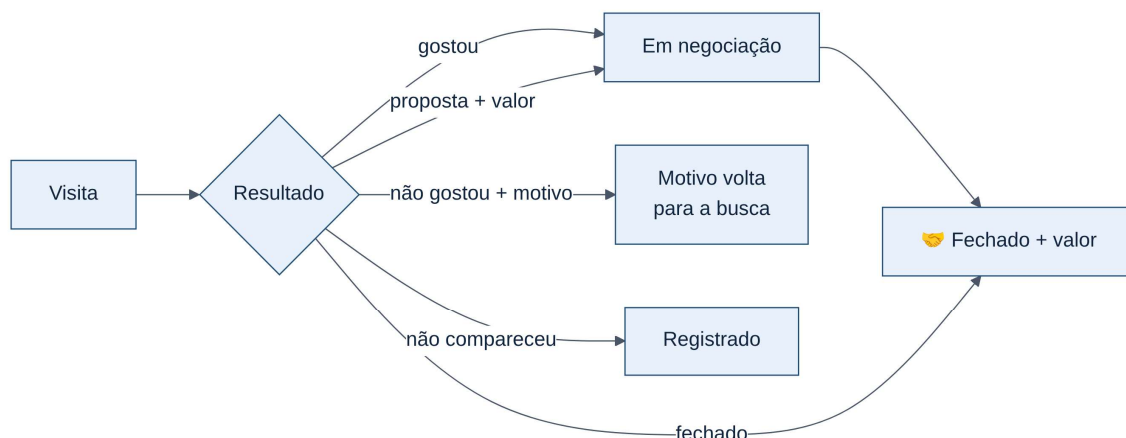


Figura 24 — O resultado da visita alimenta o funil, o Painei e as próximas sugestões ao cliente.

6.6 Painei da Imobiliária

O gestor acompanha a operação inteira em uma tela, com dados lidos direto do banco e dos eventos registrados a cada conversa: indicadores do período, funil, itens que pedem atenção, demanda × oferta por bairro (onde falta imóvel), ranking de corretores e métricas do próprio agente de IA, como satisfação e mensagens até agendar.



Painel da Imobiliária

Visão do gestor: como está a operação, onde perdemos negócio e o que precisa de atenção agora.

Período

☐ Últimos 7 dias ☐ Últimos 30 dias ☒ Tudo

Leads novos

17

Qualificados pela IA [?]

82%

Qualificado → visita

71%

Visita → negócio

43%

Valor fechado [?]

R\$ 10,4 mil

< 1 s

1ª resposta (mediana) [?]

Até agendar (mediana) [?]

0 min

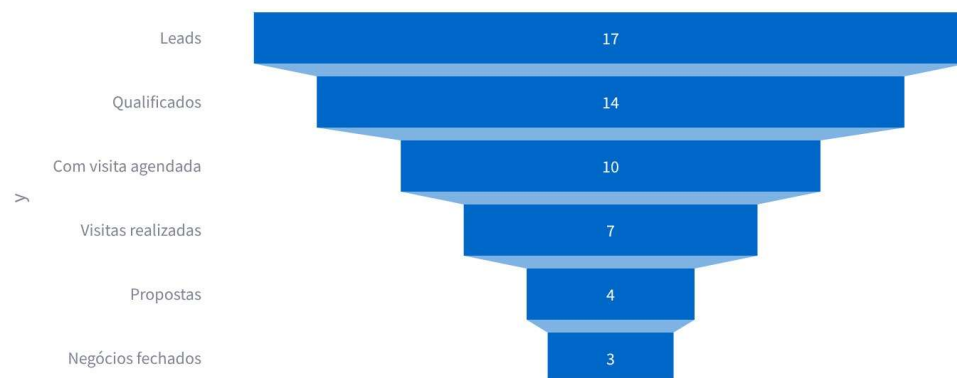
Captações [?]

1

Figura 25 — Indicadores do período.

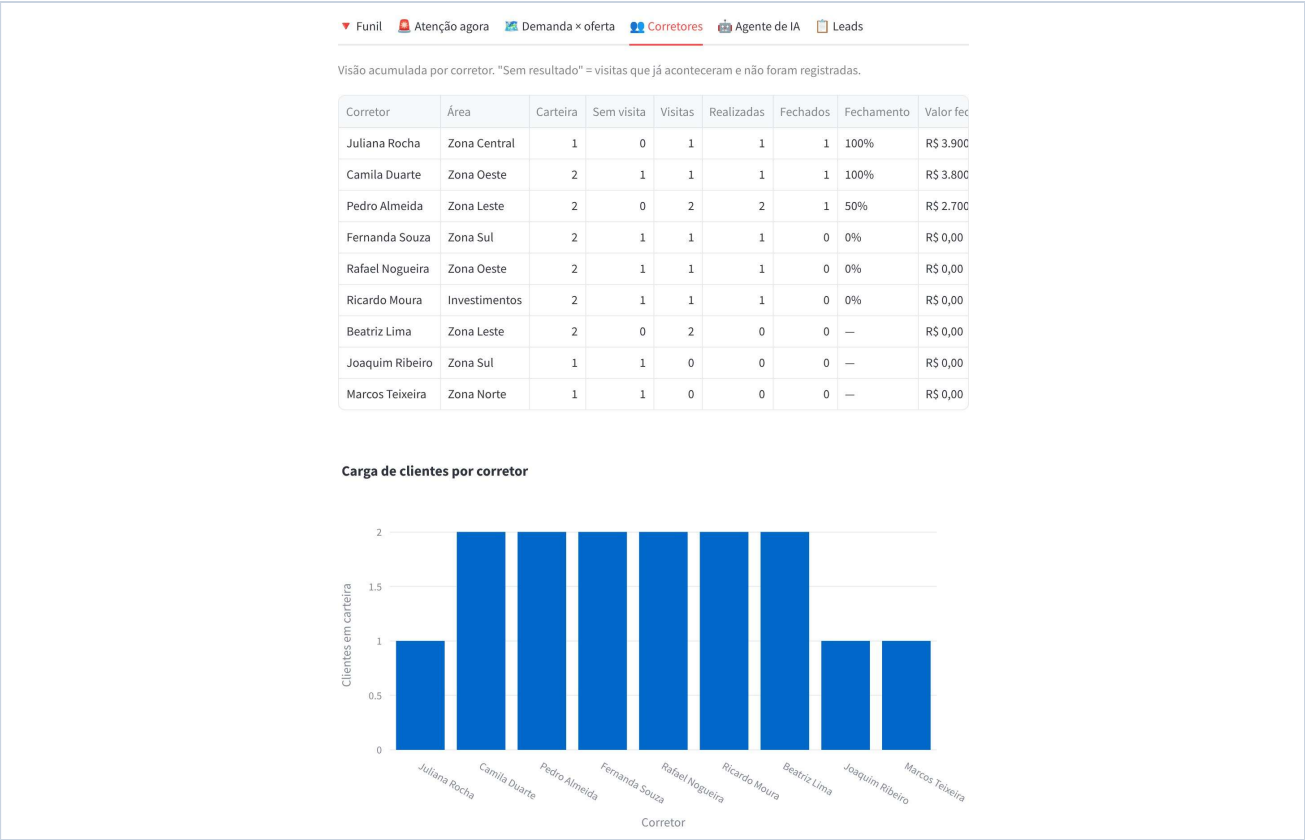
▼ Funil Atenção agora Demanda x oferta Corretores Agente de IA Leads

Funil da operação (todos os corretores)



Maior perda: depois de **Qualificados** (4 ficaram pelo caminho).

Figura 26 — Funil da operação e a etapa com a maior perda.



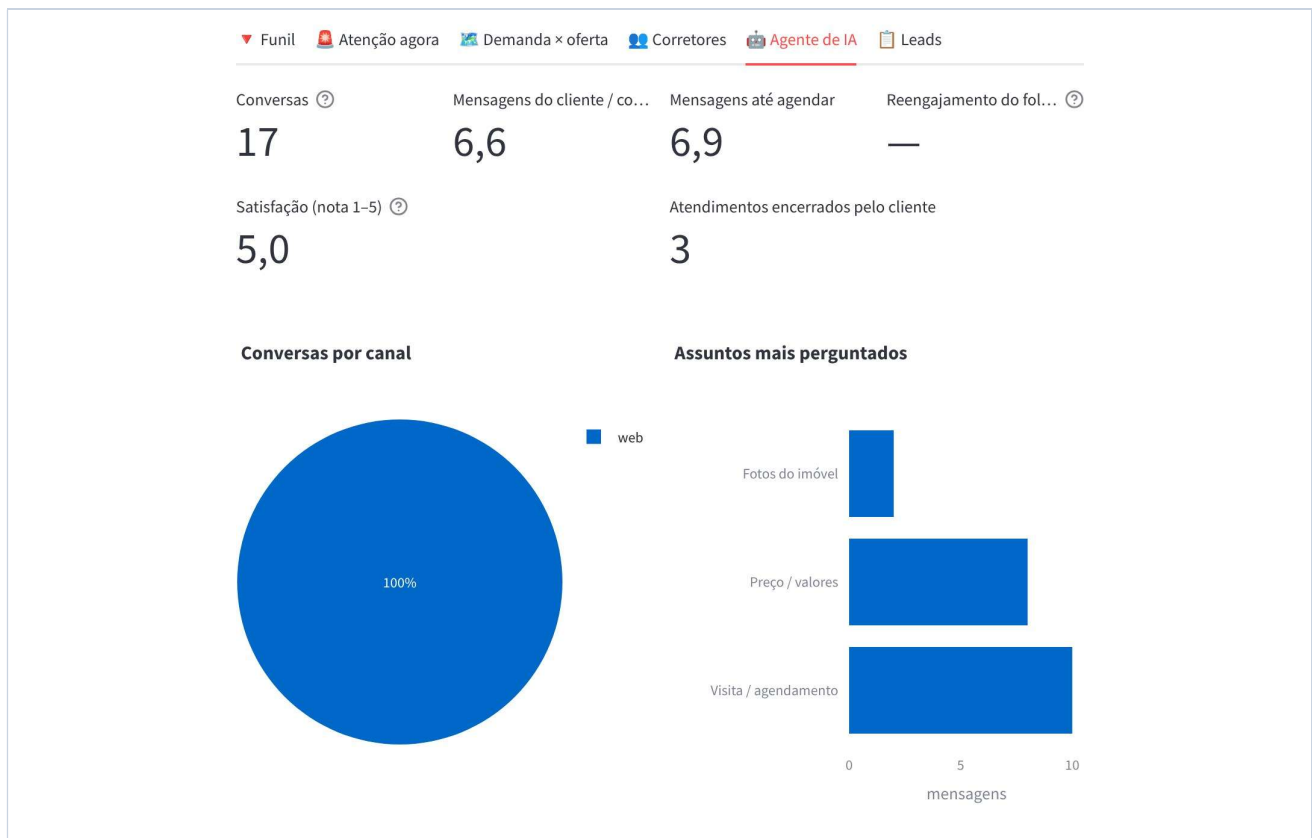


Figura 29 — Métricas do agente de IA.

6.7 Canal Telegram

O bot do Telegram usa o mesmo **ConversationService** do chat web, então todas as regras valem nos dois canais e o cliente identificado pelo CPF continua a mesma conversa em qualquer um deles. Pelo Telegram, o cliente recebe fotos dos imóveis, pode mandar áudio e recebe os avisos ativos: follow-up, pós-visita, imóveis novos compatíveis e alterações feitas pelo corretor.

Comando	O que faz
<code>/start</code>	Inicia ou retoma o atendimento; o cliente que já se identificou vê as visitas marcadas
<code>/novo</code>	Começa uma nova conversa
<code>/encerrar</code>	Encerra o atendimento com resumo e pedido de nota
<code>/followup</code>	Dispara o follow-up na hora (útil na demonstração)
<code>/posvisita</code>	Dispara o pós-visita na hora (útil na demonstração)

6.8 Voz

A voz usa a API REST do Azure Speech, sem SDK nativo. O áudio do Telegram (OGG/Opus) e o do navegador (WAV, convertido para 16 kHz mono em Python) viram texto; a resposta pode voltar falada com a voz neural **pt-BR-AntonioNeural**. Se algo falhar (chave, região, formato), o motivo é explicado em português e o cliente é convidado a escrever.

7. Modelos utilizados

O projeto combina modelos de linguagem, modelos de embeddings, serviços cognitivos e algoritmos determinísticos. Cada um foi usado onde faz mais sentido:

Área	Modelo/Técnica	Tipo	Entrada principal	Finalidade
Conversa	Azure OpenAI gpt-4.1-mini	LLM em nuvem	Mensagem + fatos calculados	Interpretar e redigir respostas, resumos e follow-ups
Conversa (testes)	MockLLMProvider	Simulador por regras	Mensagem	Executar e testar sem chave e sem custo
Extração	LLM com saída JSON + regras	Extração estruturada	Mensagem e histórico	Intenção, região, quartos, preço, urgência
Orquestração	LangGraph (11 nós)	Grafo de estados	Estado da conversa	Decidir qual agente age a seguir
Busca	paraphrase-multilingual-MiniLM-L12-v2	Embeddings locais	Texto do pedido e dos imóveis	Busca semântica (RAG)
Busca (reserva)	TF-IDF (scikit-learn)	Vetorização clássica	Texto	RAG sem modelo neural
Busca exata	SQL parametrizado	Consulta estruturada	Perfil do lead	Filtrar por negócio, região, quartos e preço
Voz	Azure Speech (STT)	Serviço cognitivo	Áudio OGG/WAV	Transcrever a fala do cliente
Voz	Azure Speech (TTS) pt-BR-AntonioNeural	Voz neural	Texto da resposta	Responder em áudio
Identidade	Dígito verificador do CPF	Algoritmo determinístico	CPF digitado	Validar e reconhecer o cliente
Avaliação	Comparáveis + R\$/m² do bairro	Estimativa estatística	Ficha do imóvel captado	Faixa de valor preliminar
Investimento	Rentabilidade × Selic/CDI	Cálculo financeiro	Preço, aluguel estimado, mercado	Comparar retorno do imóvel
Pós-visita	Classificação por padrões	Regras explicáveis	Resposta do cliente	Motivo de perda (preço, tamanho...)

8. Preparação dos dados e engenharia de prompt

Diferente de um projeto de classificação, este agente não exigiu treinamento nem fine-tuning de modelos. O trabalho equivalente ao "treinamento" foi preparar a **base de conhecimento** que o agente consulta e escrever os **prompts** de cada agente de forma que o modelo só use fatos fornecidos.

8.1 Base de imóveis e dados de mercado

- **41 imóveis** fictícios em São Paulo (`data/imoveis.json`), com tipo, negócio, bairro, zona, quartos, suítes, vagas, metragem, preço, condomínio, descrição e fotos ilustrativas. Na primeira execução, o arquivo é carregado em SQLite (`data/imoveis.db`).
- **59 bairros** com zona e vizinhança (`data/bairros_sp.json`), usados para buscar primeiro no bairro pedido e depois nos vizinhos.
- **9 corretores** com zonas de atuação (`data/corretores.json`), incluindo um especialista em investimentos.
- **Indicadores de mercado** (`data/mercado_investimento.json`): Selic, CDI, IPCA, IGP-M e R\$/m² por bairro (FipeZAP), com data de referência e fonte, atualizados manualmente.

8.2 Embeddings e RAG

Os textos dos imóveis são transformados em embeddings com o modelo multilíngue `paraphrase-multilingual-MiniLM-L12-v2`, que roda localmente e é bom em português. Os vetores ficam em cache em disco (`data/embeddings_cache.json`) para não serem recalculados a cada execução. Se o modelo não estiver disponível, o TF-IDF assume automaticamente.

8.3 Engenharia de prompt

- **Prompts curtos por agente**, com papel, tom e limites claros ("use só os fatos fornecidos", "não invente imóveis").
- **Contexto montado em código**: o prompt recebe o perfil já extraído, os imóveis já filtrados e os números já calculados.
- **Histórico recente** para manter o fio da conversa, sem marcadores internos e com o CPF mascarado.
- **Temperatura por tarefa**: baixa para extração e agenda; mais alta para saudações e follow-up.
- **Follow-up de retomada**: prompt próprio que recebe argumentos verdadeiros da base e pede uma mensagem persuasiva, sem inventar.

Exemplo real, o prompt de sistema do agente de follow-up (`src/agents/followup_agent.py`):

```
Você é {nome do agente}, um agente imobiliário (SDR) humanizado reengajando um lead que parou de responder. Escreva uma mensagem curta (1-2 frases), simpática e sem soar robótica, retomando EXATAMENTE o ponto em que a conversa parou (ex.: a última pergunta feita, o imóvel mostrado, o horário proposto) e convidando o lead a continuar. Use o primeiro nome do lead se souber. Não invente imóveis, valores nem horários.
```

O prompt define papel, tamanho, tom e limite explícito ("não invente"). Os fatos — o ponto em que a conversa parou e os argumentos da base, como quantos imóveis combinam com a busca, o retorno do melhor imóvel para o investidor ou quantos clientes procuram imóvel na região do proprietário — entram como contexto montado em código. Assim o modelo escreve bem, mas não tem de onde tirar informação falsa.

8.4 Proteção de dados no contexto do modelo

- Antes de qualquer envio ao LLM, o histórico passa por `mascarar_cpf`, que troca qualquer CPF pelo marcador `[CPF]`.
- A identificação e a validação do CPF (dígitos verificadores oficiais) acontecem só em código; o modelo nunca decide quem é o cliente.
- Marcadores internos usados pelas telas (listas, fotos, estados pendentes) são removidos do histórico antes de ir ao modelo.
- Chaves e tokens só existem no `.env`; o app recusa iniciar se encontrar segredo em arquivo versionado.

Justificativa dos dados fictícios

Uma base real de imóveis e clientes envolveria dados pessoais e comerciais de terceiros. Os dados fictícios permitiram validar o funcionamento técnico de ponta a ponta sem riscos de privacidade. Como toda a leitura passa por repositórios com interfaces, trocar a base fictícia por uma base real é uma mudança localizada.

9. Resultados e avaliação

A avaliação foi feita requisito por requisito, com testes automatizados que percorrem as conversas de ponta a ponta. Os nove objetivos e os três cenários do enunciado estão atendidos:

Objetivo	Status	Como foi atendido	Evidência (testes)
Atender leads automaticamente	☑	Grafo LangGraph + ConversationService, web e Telegram, 24 h	test_graph.py, test_telegram_bot.py
Conversa humanizada	☑	Persona, nome do cliente, reconhecimento de quem volta, voz	test_memoria_conversa.py, test_identification_agent.py
Qualificar clientes	☑	Perfil + temperatura frio/morno/quente; pergunta só o que falta	test_qualifier_agent.py, test_cenarios_desafio.py
Identificar intenção	☑	Compra, aluguel, investimento e também captação	test_cenarios_desafio.py, test_captacao.py
Coletar informações	☑	Região, quartos, orçamento, urgência; ticket e retorno; nome e CPF	test_cenarios_desafio.py, test_cpf.py
Follow-up automático	☑	Rotina no bot, /followup, limites, opt-out, argumentos reais	test_followup.py
Agendar visitas	☑	Linguagem natural, corretor por zona e carga, remarcar e cancelar, sem duplicar	test_cliente_gerencia_agenda.py, test_sem_agendamento_duplicado.py
Base simulada de imóveis	☑	41 imóveis em SQLite, fotos, vizinhança, RAG, mercado	test_sqlite_property_repository.py, test_investimento.py
Resumos para corretores	☑	Resumo ao confirmar a visita, no Painel e na Área do Corretor	test_cenarios_desafio.py

Cenário	Resultado (testado de ponta a ponta)
Compra	Entende a intenção, pergunta quartos, faixa de preço e urgência, sugere imóveis e confirma a visita com o corretor da zona.
Investimento	Identifica o investidor, o ticket e o retorno esperado, mostra a análise com dados de mercado e direciona ao especialista.
Follow-up	Depois do tempo de inatividade, retoma pelo nome e no ponto em que a conversa parou; no Telegram, a mensagem chega ao chat do cliente.

9.1 Critérios de avaliação do Tech Challenge

Critério	Situação	Destaques
Arquitetura	☑ / ●	Clean Architecture, SOLID, composition root, 11 nós especializados, duas interfaces sobre o mesmo núcleo. Escalabilidade pronta por interfaces, mas a POC roda em uma instância.
Inteligência Artificial	☑	Respostas a partir de fatos calculados, RAG, memória entre sessões e canais, voz, feedback que muda as sugestões.
Experiência do usuário	☑	Três telas para três públicos, Telegram com áudio, listas claras, confirmação antes de cancelar, erros explicados em português.
Inovação	☑	Captação com estimativa de valor, match imóvel × clientes, demanda × oferta, pós-visita que realimenta a busca, follow-up com argumentos.

É importante interpretar esses resultados com cuidado. Os testes automáticos rodam com o provedor simulado e verificam **o comportamento do sistema** — rotas, regras, cálculos, dados e integrações — e não a qualidade de redação do modelo real. A redação com o Azure OpenAI é conferida manualmente na demonstração. Também não houve teste com clientes reais nem medição de conversão em uma imobiliária.

10. Pontos importantes da solução

O Python decide, o LLM conversa: preços, horários, imóveis, corretor e cálculos saem do código. O modelo não tem como inventar um imóvel ou um horário livre.

Clean Architecture e SOLID: o domínio não depende de banco, Azure ou telas; agentes recebem interfaces; o `container.py` é o único lugar que escolhe implementações. Trocar o LLM, o banco ou o CRM é escrever uma classe nova.

Multiagentes com roteamento explícito: cada etapa tem um agente com a técnica certa, e o caminho de cada mensagem é previsível, auditável e testável.

Memória entre sessões e canais: o cliente identificado pelo CPF continua a conversa na web ou no Telegram, mesmo depois de o programa reiniciar.

Resiliência da demonstração: sem chave, o projeto roda com o provedor simulado; se o Azure cair, a Área do Corretor continua abrindo; se o arquivo do CRM simulado for corrompido, ele se recupera sozinho.

Ciclo completo do negócio: além de atender, o sistema acompanha a visita, aprende com o resultado, capta imóveis novos e mostra ao gestor onde falta oferta.

Segurança da configuração: segredos só no `.env`; um segredo encontrado em arquivo versionado impede o app de subir.

Rastreabilidade: cada mensagem e cada ação relevante geram eventos estruturados, usados pelo Painel e úteis para auditoria.

Testabilidade: 240 testes rodam em cerca de 20 segundos, sem rede e sem custo, cobrindo do CPF à abertura das três telas.

11. Pontos críticos, limitações e riscos

- **Dados fictícios:** a base de imóveis, os corretores e o CRM são simulados. A evolução mais importante é integrar uma base e um CRM reais.
- **Qualidade da redação real:** os testes usam o provedor simulado; a qualidade do texto com o Azure é verificada na demonstração, não automaticamente.
- **Alucinação residual:** mesmo com os fatos vindo do Python, o modelo ainda pode errar o tom ou omitir um detalhe. Os dados críticos (preço, horário, corretor) ficam protegidos por virem montados em código.
- **Dependência de serviços externos:** Azure OpenAI e Azure Speech dependem de internet, credenciais, custo e disponibilidade.
- **Privacidade e LGPD:** CPF, telefone e conversas são dados pessoais. Uma implantação real exigiria consentimento, controle de acesso, criptografia, política de retenção e direito de exclusão.
- **Follow-up excessivo:** mensagens demais podem incomodar. Há limite por cliente e opt-out, mas a frequência ideal precisa ser calibrada com a operação real.
- **Estimativa de valor:** a faixa calculada na captação é indicativa e não substitui a avaliação presencial nem um laudo.
- **Escala:** SQLite, sessões em JSON e rotinas dentro do bot atendem uma instância. Para produção: banco gerenciado, sessões no banco e rotinas em serviço agendado.
- **Autenticação:** a Área do Corretor e o Painel não têm login; o corretor escolhe o próprio nome numa lista.
- **Responsabilidade humana:** o agente apoia o atendimento; negociação, proposta e fechamento continuam com o corretor.

Hoje (POC)	Em produção (Azure)
SQLite local	Azure SQL ou Cosmos DB, com CPF indexado
Sessões do Telegram em JSON	Sessões no banco
Rotinas automáticas dentro do bot	Azure Functions ou Container Apps Jobs
CRM simulado	CRM real via API (RD Station, Pipedrive)
Índice vetorial em memória	Azure AI Search
Login simplificado do corretor	Autenticação e perfis de acesso
Web e Telegram	Web, Telegram e WhatsApp Business

11.1 Plano de implantação em fases

1. **Piloto controlado (1 a 3 meses):** base real de imóveis e corretores de uma unidade, CRM real via API, Azure SQL, login na Área do Corretor e no Painel e termo de consentimento LGPD no início da conversa. Metade dos anúncios vai para o agente e metade para o atendimento atual, e os indicadores são comparados.
2. **Operação (3 a 6 meses):** rotinas de follow-up e pós-visita em Azure Functions, sessões no banco, Azure AI Search para o RAG, WhatsApp Business como canal principal e alertas sobre falhas e custos.
3. **Escala (6 meses em diante):** várias unidades e equipes, calibração das regras (limites de follow-up, pesos da temperatura) com os dados do piloto, avaliação automática da redação do modelo e integração com portais de anúncio.

11.2 LGPD e responsabilidade

Tema	Tratamento na POC e o que falta para produção
Base legal e consentimento	POC usa dados fictícios. Em produção: aviso de privacidade e consentimento no primeiro contato
Minimização	O agente pede só o necessário (nome, CPF, perfil de busca); CPF mascarado no que vai ao modelo
Direito de oposição	Opt-out já implementado: "não quero mais contato" interrompe o follow-up
Retenção e exclusão	A definir com a imobiliária: prazo de guarda das conversas e exclusão a pedido
Segurança	Segredos só no .env. Em produção: criptografia em repouso, controle de acesso por perfil e registro de acesso
Transparência	O agente se apresenta pelo nome configurado; em produção, deixar explícito que é um assistente virtual

12. Testes e qualidade

Os testes automatizados são executados com `pytest` e cobrem as partes essenciais para reduzir regressões: os três cenários do enunciado de ponta a ponta, conversas reais reproduzidas, CPF, valores e datas, busca por bairro e vizinhança, RAG e investimento, o ciclo completo da agenda com avisos dos dois lados, follow-up, pós-visita, match, desempenho, captação, painel, configuração, voz, compatibilidade com versões novas do LangGraph e resiliência com o LLM fora do ar.

240 testes passando	39 arquivos de teste	~11 mil linhas em src/	~20 s para rodar tudo
-------------------------------	--------------------------------	----------------------------------	---------------------------------

- Executar toda a suíte: `python -m pytest -q`
- LLM simulado (`MockLLMProvider`): os testes não usam rede nem geram custo.
- Rede simulada com `httpx.MockTransport` para Azure Speech e Telegram.
- Bancos SQLite temporários por teste: o banco real do app não é usado.
- Teste de fumaça da interface com `streamlit.testing.AppTest`: as três telas abrem sem erro.
- Chamadas reais conferidas à parte: `scripts/diagnostico_conexao.py` e `scripts/testar_voz.py`.

12.1 Problemas encontrados e resolvidos

Problema	Solução
Follow-up "não disparava"	Primeira verificação ~20 s após subir o bot, com logs no terminal
Sugestão de Santana para quem pediu Mooca	Bairro primeiro, depois vizinhos (com pergunta), depois a zona
"Quero mais fotos" virava agendamento	Pedido de fotos tratado como detalhe do imóvel mostrado
Erro do LangGraph novo (nó com nome de chave do estado)	Nó renomeado para <code>captacao_imovel</code> + teste de regressão
Bot do Telegram caía com <code>Timeout</code> ao subir	Retentativas, timeouts maiores e mensagem clara
Área do Corretor quebrava sem acesso ao Azure	Fábrica de LLM recebe a configuração do container; saudação padrão se o LLM falhar
"Sim" depois de remarcar criava visita duplicada	Agendador confere se o cliente já tem visita no mesmo horário + teste
Arquivo do CRM simulado ficou pela metade com um processo interrompido	Gravação atômica e recuperação automática do arquivo + testes
Chave podia ir para o Git	Três camadas de configuração; segredo em TOML bloqueia a inicialização

13. Como executar o projeto

O passo a passo completo, com solução de problemas comuns, está no [README.md](#).

13.1 Pré-requisitos

- Python 3.11 ou mais novo (desenvolvido com o 3.13)
- pip e ambiente virtual
- Docker e Docker Compose, opcionais
- Conta Azure com Azure OpenAI e Azure Speech, opcional (sem ela, o projeto roda no modo simulado)
- Bot do Telegram criado no BotFather, opcional

13.2 Instalação local

```
python -m venv .venv
.venv\Scripts\activate (Linux/Mac: source .venv/bin/activate)
pip install -r requirements.txt
copy .env.example .env (Linux/Mac: cp .env.example .env)
```

Sem nenhuma chave no `.env`, o projeto já funciona com o LLM simulado. Para a IA real, preencha `AZURE_OPENAI_API_KEY`; para voz, `AZURE_SPEECH_KEY`; para o bot, `TELEGRAM_BOT_TOKEN`.

13.3 Interface Streamlit

```
python -m streamlit run app.py
```

Abre em <http://localhost:8501>, com as telas Conversar com o Agente, Painel da Imobiliária e Área do Corretor.

13.4 Bot do Telegram

```
python telegram_bot.py
```

Comandos: `/start`, `/novo`, `/encerrar`, `/followup` e `/posvisita`. No Windows, `iniciar_tudo.bat` sobe a interface web e o bot de uma vez.

13.5 Docker

```
docker compose up --build
```

- Imagem `python:3.11-slim` com PyTorch só para CPU e o modelo de embeddings já baixado no build, para a primeira busca não esperar o download.
- Um serviço, `agente-sdr`, na porta 8501, com o `.env` como arquivo de ambiente e a pasta `data/` montada como volume (os dados sobrevivem a novas versões do container).
- Verificação de saúde em `/_stcore/health` e reinício automático (`restart: unless-stopped`).

13.6 Testes e diagnóstico

```
python -m pytest -q
python scripts/diagnostico_conexao.py
python scripts/testar_voz.py
```

14. Conclusão

O desenvolvimento deste Tech Challenge permitiu aplicar, em um único projeto, conceitos de IA Generativa, orquestração multiagente, RAG, memória conversacional, serviços cognitivos de voz e arquitetura de software. O principal aprendizado foi perceber que a qualidade de um agente conversacional não depende só do modelo de linguagem, mas principalmente do que fica **em volta** dele: a separação entre o que o código decide e o que o modelo escreve, os dados que ele consulta, os limites que ele respeita, os testes e a forma como cada público enxerga a informação.

A arquitetura criada atende ao objetivo acadêmico de demonstrar um Agente SDR imobiliário de ponta a ponta. Ao mesmo tempo, o projeto deixa claros os limites entre uma POC funcional e um produto: dados reais, CRM real, autenticação, banco gerenciado e avaliação da redação com usuários seriam os próximos passos.

Considero que a regra "**o Python decide, o LLM conversa**" foi a parte mais relevante da entrega, porque ela é o que permite confiar no agente. Em vez de um chatbot que responde bem mas pode inventar um preço ou um horário, o Sr. Agim conversa de forma natural e, ao mesmo tempo, só oferece o que existe na base, no horário que está livre e com o corretor certo. O corretor continua no centro do negócio; o agente garante que nenhum cliente esfrie esperando resposta.

15. Referências

15.1 Documentação do projeto

Todos os sete documentos Markdown (.md) do projeto. Os links apontam para o **repositório no GitHub** e abrem de qualquer lugar, sem precisar baixar o projeto.

Documento	Conteúdo
README.md	Instalação, como executar (só web, só Telegram, os dois ou Docker), configuração, uso por perfil, estrutura de pastas, testes e problemas comuns
docs/contexto_projeto.md	O desafio, o problema de negócio, os públicos, o escopo, premissas e glossário
docs/decisoese_projeto.md	As 21 decisões técnicas no formato ADR: contexto, decisão, alternativas e consequências
docs/ARQUITETURA.md	Camadas, agentes e todos os fluxos em 25 diagramas
docs/SOLID.md	Clean Architecture e os 5 princípios SOLID no código, com arquivos, classes e trechos reais
docs/relatorio_tecnico.md	Tecnologias, aplicação da IA, funcionalidades, testes, resultados e limitações
docs/VALIDACAO_REQUISITOS.md	Requisito por requisito e critério por critério, com os testes que comprovam cada um

15.2 Referências técnicas

- Projeto desenvolvido: <https://github.com/fmonin/tech-fase5-agente-sdr-imobiliario-ia>
- LangGraph: <https://langchain-ai.github.io/langgraph/>
- LangChain: <https://python.langchain.com>
- Microsoft Azure OpenAI Service: <https://learn.microsoft.com/azure/ai-services/openai/>
- Microsoft Azure AI Speech: <https://learn.microsoft.com/azure/ai-services/speech-service/>
- Sentence-Transformers: <https://www.sbert.net>
- Modelo paraphrase-multilingual-MiniLM-L12-v2: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>
- scikit-learn (TF-IDF): <https://scikit-learn.org>
- Streamlit: <https://streamlit.io>
- python-telegram-bot: <https://python-telegram-bot.org>
- SQLite: <https://www.sqlite.org>
- pytest: <https://docs.pytest.org>
- Índice FipeZAP: <https://www.fipe.org.br/pt-br/indices/fipezap/>
- Banco Central do Brasil — Taxa Selic: <https://www.bcb.gov.br/controleinflacao/taxaselic>
- OLDROYD, J.; McELHERAN, K.; ELKINGTON, D. The Short Life of Online Sales Leads. *Harvard Business Review*, mar. 2011: <https://hbr.org/2011/03/the-short-life-of-online-sales-leads>
- Preços do Azure OpenAI: <https://azure.microsoft.com/pricing/details/cognitive-services/openai-service/>
- Preços do Azure AI Speech: <https://azure.microsoft.com/pricing/details/cognitive-services/speech-services/>
- MARTIN, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

Anexo A — Decisões de projeto

Resumo das 21 decisões. O registro completo, com alternativas e consequências, está em docs/decisoes_projeto.md.

#	Decisão	Por quê
D01	Clean Architecture com SOLID	Testar e trocar peças (LLM, banco, CRM) sem efeito cascata
D02	Multiagentes com LangGraph	Prompts curtos e fluxo previsível, controlado por Python
D03	O Python decide, o LLM conversa	Nada de preço, horário ou imóvel inventado
D04	Identificação por CPF sem LLM	Identidade validada e mascarada; um cliente, uma ficha
D05	Azure OpenAI com provedor simulado	IA real na demo, custo zero para quem avalia
D06	SQL primeiro, RAG como rede de segurança	Exatidão no caso comum, flexibilidade no vago
D07	Embeddings locais e gratuitos	RAG sem custo e sem rede
D08	SQLite atrás de interfaces	Zero configuração, troca simples por banco gerenciado
D09	Memória no Lead, estado descartável	Contexto preservado entre reinícios e canais
D10	Streamlit + Telegram no mesmo núcleo	Uma regra vale nos dois canais
D11	Corretor por zona e carga	Distribuição justa e coerente com o território
D12	Follow-up com limite, argumentos e opt-out	Insistência na medida, com respeito ao cliente
D13	Agentes de entrada antes da qualificação	Agenda, captação e encerramento tratados no lugar certo
D14	Marcadores invisíveis nas mensagens	Fotos, listas e estados pendentes sem tabelas extras
D15	Configuração em três camadas	Segredos só no .env; segredo em TOML impede a subida
D16	Voz pela API REST do Azure Speech	Sem dependência nativa; testável sem rede
D17	Captação como fluxo próprio	Fecha o ciclo demanda ↔ oferta
D18	Resultado da visita realimenta a busca	A conversa aprende com a visita
D19	Rotinas automáticas dentro do bot	Funciona sem infraestrutura extra na POC
D20	Testes sem rede e sem o banco real	Rápidos, reproduzíveis e gratuitos
D21	Interface separada por tela	Cada tela evolui sem conflito

Anexo B — Estrutura de pastas

```
Imobiliária Tech fase 5/
├── app.py                interface web (python -m streamlit run app.py)
├── telegram_bot.py       bot do Telegram (python telegram_bot.py)
├── iniciar_tudo.bat      Windows: sobe os dois de uma vez
├── README.md            instalação, execução e uso
├── config/
│   ├── settings.toml    configurações do projeto (vai para o Git)
│   └── settings.local.toml ajustes só da máquina (fora do Git)
├── .env.example         modelo do .env (só chaves e tokens)
├── data/               imóveis, imagens, corretores, bairros, mercado
├── src/
│   ├── config.py        junta settings.toml + settings.local + .env
│   ├── container.py     monta e injeta todas as dependências
│   ├── domain/          entidades e regras puras
│   ├── agents/          agentes do grafo e do corretor
│   ├── services/        casos de uso
│   ├── infrastructure/  LLM, RAG, SQLite, Telegram, voz, CRM, logs
│   └── interface/       Streamlit: navegação e uma tela por arquivo
├── scripts/            follow-up manual, diagnóstico, teste de voz
├── tests/              240 testes automatizados (pytest)
└── docs/               contexto, decisões, arquitetura, SOLID,
                       relatório técnico e validação
```

Anexo C — Mapa de componentes

Componente	O que faz	Usa LLM?
IdentificacaoAgent	Cadastra ou reconhece o cliente (nome + CPF) e mostra as agendas	não
EncerramentoAgent	Encerra o atendimento, pede a nota, registra o opt-out	não
AgendaClienteAgent	Cliente vê, cancela e remarca; captura o feedback do pós-visita	não
CaptacaoImovelAgent	Cadastro do imóvel do proprietário, estimativa e avaliação	não
QualificadorAgent	Extraí os dados da mensagem e classifica a temperatura	sim
EsclarecedorAgent	Pergunta, com naturalidade, o que ainda falta	sim
ConsultorImoveisAgent	Busca (SQL + RAG) e apresenta imóveis e investimento	sim
DetalheImovelAgent	Ficha completa de um imóvel, com fotos	sim
AgendadorAgent	Escolhe o corretor, evita conflito e duplicidade, registra a visita	sim
ResumidorAgent / RecapituladorAgent	Resumo para o corretor / "o que já conversamos?"	sim
MenuCorretorAgent	Assistente do corretor com 9 opções	redige
FollowUpAgent + ArgumentosFollowUp	Reengaja leads parados com fatos reais	sim
ConversationService	Orquestra uma rodada: estado → grafo → lead → CRM → eventos	—
AgendaService	Cancelar, remarcar e avisos entre cliente e corretor	—
PosVisita / Captação / Match	Resultados de visitas e captações; imóvel novo × clientes	—
Desempenho / PainelGestor	Funil do corretor e visão do gestor	—
AzureSpeechService	Voz ↔ texto	—
AppContainer	Monta e injeta todas as dependências	—